Cankaya University

FACULTY OF ENGINEERING

COMPUTER ENGINEERING DEPARTMENT

FINAL REPORT

CENG 407-408

Innovative System Design and Development

A SMART HOME SYSTEM

Group Members:

İbrahim Ersan Özdemir - 202211054

Deniz Arda Çınarer - 202211019

Duhan Ayberk Seven - 202211062

Enis Mirsad Şengül – 202211065

Advisor: Prof. Dr. Murat Koyuncu

Introduction

The proliferation of the Internet of Things (IoT) has transitioned from a futuristic concept to a practical, everyday reality, fundamentally altering human interaction with physical environments. This technological shift has catalyzed the rapid development and adoption of smart home systems. In the contemporary market, commercial platforms such as Google Home, Amazon Alexa, and Apple HomeKit have popularized the concept of home automation, primarily focusing on convenience and voice-activated controls.

However, despite their prevalence, many existing commercial systems and homes advertised as "smart" often lack the comprehensive functionalities that users expect. These systems frequently operate as closed ecosystems with limited capabilities, prioritizing user convenience over holistic security, in-depth environmental monitoring, and intelligent, proactive automation. A significant gap exists between systems that offer simple, command-based automation and the growing need for integrated platforms that intelligently manage a home's safety, security, and environment.

The challenge lies not only in connecting devices but in creating a cohesive system that can sense, process, and act upon complex and critical information. Many platforms fail to adequately address the fundamental security and safety concerns of homeowners, such as the immediate detection of environmental threats like fire, flooding, or unauthorized intrusion. Furthermore, while a system may alert a user to motion, it often lacks the intelligence to differentiate a benign event (a resident returning home) from a genuine threat (an intruder). This limitation places the burden of analysis and response entirely on the end-user.

This project, "A Smarthome System", is designed to address these limitations and bridge this functional gap. The project proposes the design and development of a comprehensive, multi- faceted smart home system that integrates environmental monitoring, advanced security, and intelligent automation using IoT technologies.

The core of the proposed system is architected around three essential, interconnected modules:

1. **An IoT Sensor Module** responsible for real-time data acquisition from the home environment, utilizing a suite of sensors for heat, humidity, smoke, water, and images.

2. **A Cloud-Based Web Application** serving as the central nervous system, responsible for receiving, processing, and storing the collected data, and presenting it to users.
3. **A Mobile Application** designed to provide end-users with uninterrupted, remote monitoring capabilities and instant notifications.

A distinguishing feature of this project is the deep integration of artificial intelligence (AI) for security analysis. The system will not merely report motion; it will actively process captured image data to identify human movement, compare it against a database of known residents, and intelligently identify unauthorized individuals entering the home. This moves the system from a passive reporter to an active participant in home security.

This endeavor aligns with broader technological and societal goals, specifically contributing to the UN Sustainable Development Goal 9: "Industry, Innovation, Technology, and Infrastructure". By focusing on research and development in the increasingly vital fields of IoT, AI, and data management, this project aims to enhance technological capabilities and foster innovation.

Project Objectives

To achieve the vision laid out in the introduction, the project is guided by one primary goal, which is broken down into several specific, measurable sub-objectives.

1.1. Main Objective

The main objective of this project is to design and develop a robust, integrated smart home system that monitors the indoor environment, ensures resident safety, and performs intelligent automation using Internet of Things (IoT) technologies.

This system is intended to provide residents with peace of mind by enabling them to monitor their homes remotely from anywhere and receive instantaneous notifications regarding critical events such as fire, flooding, or burglary.

1.2. Sub-Objectives

To successfully realize the main objective, the project will focus on the following four key sub-objectives:

- **To Implement Comprehensive Monitoring:** This involves the deployment of a robust environmental sensing network. The system must collect and process real-time data from a variety of sensors, including but not limited to temperature, humidity,

smoke, and water level detection. It must be capable of immediately and reliably notifying users in the event of emergencies like fire or flooding.

- **To Integrate Artificial Intelligence (AI) for Security:** This objective focuses on creating an intelligent security component. Upon the detection of human motion, the system will automatically capture camera footage and utilize AI processing models to analyze the visual data. The goal is to accurately distinguish between authorized residents and unauthorized persons, thereby reducing false alarms and providing actionable security alerts.
- **To Ensure Seamless User Accessibility:** This objective is to provide continuous and intuitive remote access to the home's status. This will be achieved through the development of two primary user interfaces: a comprehensive web application (Module 2) for detailed data visualization and management, and a mobile application (Module 3) for on-the-go monitoring and push notifications.
- **To Enable Action-Based Automation:** This objective aims to extend the system's functionality beyond passive monitoring into active home control. The system will be designed to enable action-based automation, allowing it to control actuators (e.g., smart curtains, air conditioning) based on predefined thresholds or rules derived from sensor data (e.g., adjusting curtains based on light intensity or activating AC based on temperature)

Scope

To ensure a clear focus and successful completion, the project's scope is defined in two parts: the core functionalities required for project completion and the extended functionalities identified as potential future work.

1.3. Core Functionality (Minimum Scope)

The successful completion of this project is contingent upon the stable and functional implementation of its three core modules and the integrated AI feature.

1. **The IoT Sensor Module (Module 1):** This foundational module is responsible for all data acquisition. The minimum scope includes the successful integration and operation of sensors for temperature, humidity, smoke, water level , and motion (PIR) , as well as a camera for visual data capture.

2. **The Cloud-Based Web Application (Module 2):** This module serves as the central backend and data hub. It must be capable of receiving all data from the IoT module, securely storing this data , and presenting both real-time and historical data to the user via a web interface.
3. **The Mobile Monitoring Application (Module 3):** This module provides the primary

interface for end-user remote interaction. It must allow users to monitor their homes from anywhere , visualize sensor data , and, most critically, receive instant push notifications for emergency events.

4. **AI-Based Recognition Feature:** A non-negotiable component of the core scope is the development and integration of a stable AI-based recognition feature. The system must successfully execute the control flow of detecting motion, capturing an image, and applying an AI model to classify the detected person as "authorized" or "unauthorized" .

1.4. Extended Scope (Potential Future Work)

While the following features are central to the project's long-term vision, they are considered outside the minimum scope required for the CENG 407 project. They represent logical extensions and future work that can be built upon the core architecture.

- **Seamless Integration of New Hardware:** Designing the architecture to be fully scalable, allowing for the easy integration of new, unplanned sensors (e.g., light sensors, sound sensors) and actuators (e.g., relays, servo motors).
- **Scenario-Based Automation Engine:** Implementing an advanced, IFTTT-style (If This, Then That) rule engine. This would allow users to create personalized "scenarios" (e.g., "Away Mode," "Night Mode") and complex automation rules, such as "IF (motion is detected) AND (AI detects unauthorized person) AND (Time is Night) THEN (Turn on all lights) AND (Send high-priority alert)".
- **Energy Consumption Monitoring:** Integrating non-invasive current transformer sensors (e.g., SCT-013) to monitor real-time power (W) and cumulative energy (kWh) usage. This would provide users with actionable insights into energy inefficiency and contribute directly to UN Sustainable Development Goals 7 and 9.
- **Advanced AI Analysis:** Expanding the AI module's capabilities beyond security to include safety and wellness features. This could involve using pose estimation models (like MediaPipe) for fall detection to ensure elderly safety or implementing a sound classification model to detect specific abnormal sounds like glass breaking or a smoke alarm.

Literature Review

The smart home ecosystem has evolved from simple remote-controlled appliances to integrated environments that use Internet of Things (IoT) architectures, edge gateways, and artificial intelligence–based automation. Recent work often describes smart homes in layered architectures such as perception, network, and application layers, where low-power wireless protocols including Wi-Fi, Bluetooth Low Energy (BLE), and Zigbee/Thread connect sensors, actuators, and controllers. Across the literature, the main research priorities are interoperability, security and privacy, and intelligent, context-aware automation under resource constraints.

Interoperability and Standardisation

The fragmentation of vendor ecosystems has made smart home deployment difficult, leading to the development of unified standards such as the Matter specification by the Connectivity Standards Alliance (CSA). Matter provides IP-based connectivity, standardised device onboarding, and local multivendor control, enabling communication between different classes of smart devices such as locks, sensors, appliances, and HVAC systems [3]. Industry reports highlight Matter's growing device support and improved developer workflows as important drivers of adoption. However, in real homes, mixed ecosystems with both Matter-certified and legacy devices are still common, and architectural guidelines for bridging these environments are still limited in the literature [3,4].

Security and Privacy Considerations

Security is a central concern in smart home design, especially as the number of devices and communication channels increases. Empirical studies on early platforms revealed vulnerabilities such as overly permissive automation rules and weak onboarding procedures that allowed unauthorised device control. More recent surveys extend the threat model to the entire data lifecycle and analyse both technical exploits and privacy risks for users [1,4]. Proposed countermeasures include hardware roots of trust, mutual authentication, encrypted communication, and capability isolation. The literature also stresses that usable security through clear user interfaces is essential for widespread adoption.

Edge-First Architecture and Resilience

Recent studies increasingly promote edge-first architectures to reduce dependence on continuous cloud connectivity. Moving sensing, analytics, and decision-making from remote clouds to local gateways reduces latency, improves privacy, and allows systems to continue operating during internet outages [1,3]. In smart homes, edge hubs can run lightweight machine learning models for occupant detection, anomaly recognition, and real-time automation, while cloud services remain responsible for long-term analytics. This design is particularly important for safety-critical tasks such as intrusion detection or fire alarms. However, maintaining secure and scalable edge infrastructures under resource constraints remains a challenge [1,3].

AI-Driven Automation and Activity Recognition

Machine learning enables advanced capabilities in smart homes, including human activity recognition, predictive scheduling, and reinforcement learning–based optimisation of HVAC and lighting systems. Deep learning models such as convolutional and recurrent neural networks are widely used for these tasks [2,3]. Key challenges include dataset collection in real homes, robustness against noisy sensor data, and efficient inference on edge devices. Several studies also emphasise the importance of human-in-the-loop mechanisms to reduce false positives and maintain user trust [2–4].

Energy Optimisation and Sustainability

Smart home systems increasingly focus on sustainability through adaptive scheduling, demand response, and predictive energy management. Review studies show that user acceptance strongly depends on visible energy savings, simple override controls, and integration with utility systems [3,4]. Edge-based controllers that work without constant cloud access can further reduce bandwidth usage and improve resilience. Nevertheless, these solutions require reliable context sensing and accurate prediction models.

Human Factors, Usability and Adoption

Human factors play a crucial role in the adoption of smart home technologies. User studies report gaps in security awareness, privacy understanding, and update practices. With the growth of multimodal interfaces, recent research recommends transparent onboarding mechanisms and configurable privacy dashboards to support user trust [2,4]. Although standards such as Matter reduce vendor lock-in, mixed ecosystems can still introduce usability challenges [3].

Synthesis and Research Gaps

Although interoperability, security, edge computing, and AI-based automation are all well studied individually, their joint evaluation in real-world deployments remains limited. There is still a lack of open benchmarks for on-device AI in realistic smart home environments. Moreover, long-term user trust in edge-based AI systems is not sufficiently explored in current studies. This project aims to address these gaps by combining interoperable frameworks, secure local analytics, and adaptive automation policies in a real-world smart home deployment [1–4].

Conclusion

In summary, the literature highlights interoperability as a core enabler, security and privacy as fundamental requirements, and intelligent automation as the main value proposition for users. Effective smart home systems must integrate these three dimensions in a unified architecture. Building on this perspective, the present study proposes an edge-centric, AI-enabled smart home framework designed for real-world evaluation.

Technology Review

This section provides a comprehensive technical analysis of the leading smart home ecosystems currently dominating the market: Samsung SmartThings, Apple HomeKit, Google Home, and Home Assistant [5]. By evaluating their underlying architectures, communication protocols, security mechanisms, and feature sets, this review aims to identify the specific strengths and architectural trade-offs of each platform. The insights gained from this comparative analysis will serve as a foundational benchmark for the architectural decisions, component selection, and privacy strategies in the development of our project [6].

1. Samsung SmartThings

1.1. System Overview

Samsung SmartThings is a mature smart home platform featuring one of the broadest device compatibilities on the market. Although initially built around a hardware "Hub," it has evolved into a hybrid ecosystem that unifies Samsung's own devices (TVs, refrigerators, etc.) and hundreds of third-party products under a single application, blending cloud and local processing [7].

1.2. Technical Architecture and Topology

SmartThings' architecture utilizes a hybrid (cloud + local) structure:

- **SmartThings Cloud:** The platform's brain resides in the cloud. Most devices (especially Wi-Fi-connected ones and third-party cloud integrations) connect to Samsung's servers to process commands, run automations, and communicate with the app.
- **SmartThings Hub:** The SmartThings Hub (or compatible hubs like Aeotec) acts as a bridge for devices communicating via local protocols (Zigbee, Z-Wave). It receives data from these devices and transmits it to the cloud [7].
- **Edge Drivers:** Samsung's newer architecture supports a system called "Edge Drivers." This allows compatible devices and automations to run locally directly on the Hub, even without an internet connection. This reduces latency and increases reliability [8].
- **Topology:** Devices (sensors, bulbs) connect to a Hub (via Zigbee/Z-Wave/Thread) or directly to the home network (via Wi-Fi/Matter). The Hub and Wi-Fi devices communicate with the SmartThings Cloud. The mobile application (client) also sends commands to this cloud.

1.3. Communication Protocols and Standards

SmartThings is one of the most flexible platforms in terms of protocol support:

- **Wi-Fi:** For standard network devices.
- **Zigbee & Z-Wave:** Supports the two most common local smart home protocols via a Hub.
- **Matter & Thread:** New-generation Hubs support the Matter standard and its underlying Thread network protocol, further expanding the ecosystem [5].
- **Bluetooth:** Generally used for the initial setup (onboarding) of devices.

1.4. Functionality and Feature Set

The platform is managed via the "SmartThings" mobile app and offers rich functionality:

- **Broad Device Support:** It has the widest third-party device support on the market, working seamlessly with non-Samsung brands (e.g., Philips Hue, Yale, Ring).
- **Automations (Routines):** Allows for the creation of powerful "If-Then" logic automations.
- **Scenes:** Groups of commands that set multiple devices to predefined states with a single touch.
- **Samsung Ecosystem:** Provides deep integration with Samsung TVs, soundbars, refrigerators, and washing machines.
- **Voice Assistant:** Compatible with Bixby (Samsung), Google Assistant, and Amazon Alexa.

1.5. Security and Data Privacy

- **Security:** The Samsung Knox platform provides security in the Hub hardware and software. Communication is encrypted [10].
- **Data Privacy:** SmartThings collects usage data to improve and personalize the service. Data is subject to Samsung's general privacy policy and may be shared with partners. Data processing largely occurs in the cloud (excluding Edge Drivers).

2. Apple HomeKit

2.1. System Overview

Apple HomeKit is a security and privacy-focused smart home framework deeply integrated into Apple's iOS, macOS, and tvOS operating systems. The ecosystem is centered around the "Home" app and the Siri voice assistant [11].

2.2. Technical Architecture and Topology

HomeKit uses a local-first and decentralized hub architecture:

- **Home Hub:** A "Home Hub" is required for HomeKit to operate. This is a role assigned via software and is executed by an Apple TV, HomePod, or HomePod mini within the home.
- **Local Processing:** All automation logic and device communication are processed locally on this Home Hub instead of being sent to the cloud. Automations continue to function even if the internet connection is lost [12].
- **Topology:** Devices (Wi-Fi, Bluetooth, Thread/Matter) communicate directly with the Home Hub over the local network. The "Home" app on an iPhone also communicates directly with the Home Hub when on the local network.
- **Remote Access:** When the user is away from home, their iPhone connects to the Home Hub via a secure tunnel through iCloud. Apple cannot see the commands; it only transports the encrypted data [11].

2.3. Communication Protocols and Standards

- **Wi-Fi & Bluetooth LE:** The initial versions of HomeKit were based on these two protocols.
- **Matter & Thread:** Apple has played a leading role in the development of the Matter standard. The HomePod mini and newer Apple TV models act as "border routers" for Thread, connecting these devices to the network [9].
- **No Zigbee/Z-Wave Support:** HomeKit does not directly support these protocols. A HomeKit-compatible bridge is required to use such devices.

2.4. Functionality and Feature Set

- **Home App:** The central application for managing all devices, scenes, and automations.
- **Siri Integration:** The platform's greatest strength is its deep integration with Siri.
- **Automations:** Automations can be set based on location, time, sensor triggers, or the status of other devices.
- **HomeKit Secure Video (HKSV):** A feature for compatible cameras. Video analysis is performed locally on the Home Hub, not in the cloud. The video is then encrypted and uploaded to iCloud in a way that even Apple cannot access [12].
- **Adaptive Lighting:** Automatically adjusts the color temperature of compatible lights throughout the day.

2.5. Security and Data Privacy

This is HomeKit's strongest area:

- **End-to-end Encryption:** All communication between the Home Hub, devices, and the user's Apple devices is end-to-end encrypted. No one, including Apple, can read the content of this data [11].
- **No Data Collection:** Apple does not collect smart home usage data for advertising or profiling purposes. Privacy is a core design principle of the platform.
- **Certification:** In the past, MFi (Made for iPhone) certification was mandatory, ensuring hardware security.

3. Google Home (Google Nest)

3.1. System Overview

Google Home is a smart home ecosystem centered on the Google Assistant, making it exceptionally strong in artificial intelligence and voice control. Initially shaped around "Google Home" speakers, the platform now encompasses a wide range of hardware under the Nest brand and thousands of third-party integrations [13].

3.2. Technical Architecture and Topology

Google's architecture, unlike Apple's, is cloud-centric:

- **Google Cloud:** The platform's brain is Google's cloud servers. Nearly all automation logic, device management, and third-party integrations run in the cloud [13].
- **Device Connection:** Nest speakers, displays, and most third-party devices connect directly to the Google Cloud via the home's Wi-Fi network.
- **Topology:** When a user says, "Hey Google, turn on the lights," this voice command is sent to Google's servers for processing. The server understands the command and sends it to the relevant device's cloud.
- **Local Processing (Limited):** With the advent of the Matter standard, new devices like the Nest Hub and Nest Wifi Pro are beginning to act as "Thread border routers," allowing some Matter commands to execute faster on the local network. However, the core architecture remains dependent on the cloud [14].

3.3. Communication Protocols and Standards

- **Wi-Fi & Bluetooth:** These are the mainstays of the ecosystem.
- **Matter & Thread:** Google is a strong proponent of Matter, and its new hardware comes with Thread support.
- **No Zigbee/Z-Wave Support:** It does not directly support local protocols like SmartThings does.

3.4. Functionality and Feature Set

- **Google Assistant:** The most advanced and conversational voice assistant on the market.
- **Routines:** Offers automations based on presence or voice commands. It is also possible to write more advanced automations using the Script Editor.
- **Nest Ecosystem:** Offers deep and seamless integration with Nest Cams, Nest Thermostats, and Nest Doorbells.
- **Cast Technology:** Highly successful in casting audio and video between devices.
- **Google Home App:** The main application for managing all devices and routines.

3.5. Security and Data Privacy

- **Security:** Google has strong mechanisms for account security (2-Step Verification, etc.). Communication is encrypted.
- **Data Privacy:** This is Google's most debated aspect. Google collects data, including voice commands and device usage data, to personalize services and (depending on user settings) for ad targeting [15].

4. Home Assistant

4.1. System Overview

Home Assistant (HA) is an open-source, free, and local-first smart home platform. It is not owned by a company; it is managed by a worldwide community of developers and users. It is known for its power, flexibility, and customization, but it is a platform that requires technical knowledge [16].

4.2. Technical Architecture and Topology

Home Assistant's architecture is fundamentally different from the other three:

- **Self-Hosted:** Home Assistant is not a cloud service. It is software that the user installs on their own hardware (Raspberry Pi, mini-PC, etc.).
- **Local-First Control:** The platform's brain is this device in the user's home. All automations, device communication, and data are processed and stored on the local network by default. The entire system continues to run even if the internet is down [16].
- **Topology:** All smart devices in the home connect directly to the server running the Home Assistant software.
- **Remote Access (Optional):** Remote access is disabled by default. Users must enable it themselves, either by using Nabu Casa or by manually setting up a VPN or a reverse proxy.

4.3. Communication Protocols and Standards

Home Assistant is superior in protocol support:

- **All Supported:** It supports Wi-Fi, Bluetooth, Matter, and Thread.
- **USB Dongle Support:** Through inexpensive USB adapters (dongles), it can control Zigbee and Z-Wave devices directly and locally [17].
- **Integrations:** With thousands (3000+) of official and community-developed "integrations," it can communicate with almost any device or service imaginable.

4.4. Functionality and Feature Set

- **Unlimited Customization:** Its greatest strength. The interface and automations are completely customizable.
    - **Powerful Automation Engine:** Allows for the creation of highly complex and conditional automation scenarios.
    - **Add-ons:** Dozens of additional services can be installed on the Home Assistant setup (e.g., AdGuard, Node-RED).
    - **Local Voice Assistant (Assist):** It has its own local voice assistant named "Assist," which does not rely on the cloud.
    - **Learning Curve:** This flexibility comes with a steep learning curve.

4.5. Security and Data Privacy

- **Maximum Privacy:** By default, no data leaves your home. Your data is not collected, analyzed, or used for advertising by any company [16].
- **User Responsibility:** The security of the platform is entirely the user's responsibility.

5. Comparative Analysis of Platforms

The following table provides a detailed comparison of the four platforms based on critical architectural and functional criteria utilizing the data gathered [5].

Feature	Samsung SmartThings	Apple HomeKit	Google Home	Home Assistant
5.1 Core Architecture	Hybrid (Cloud + Local)	Local-First	Cloud-Centric	Local (Self-Hosted)
5.1 Key Strength	Device compatibility, Samsung ecosystem	Privacy, Security, Apple ecosystem	Voice Assistant (AI), Google services	Customization, Local control, Flexibility
5.2 Setup Difficulty	Easy / Medium	Easy	Very Easy	Difficult (Technical knowledge required)
5.3 Zigbee / Z-Wave Support	**Yes** (with Hub)	**No** (Bridge only)	**No** (Bridge only)	**Yes** (with USB Dongle)
5.4 Matter / Thread Support	Yes	Yes (Strong support)	Yes (Strong support)	Yes
5.5 Data Privacy Model	Data collected (Service/Ads)	No data collected (Max privacy)	Data collected (Service/Ads)	No data collected (User control)
5.6 Offline Functionality	Partially (with Edge Drivers)	**Yes** (Fully)	Very Limited	**Yes** (Fully)

5.1. Core Architecture and Key Strengths

The fundamental design philosophy varies significantly across platforms. Samsung SmartThings employs a hybrid approach, balancing cloud connectivity with local execution via Edge Drivers. Apple HomeKit and Home Assistant prioritize a "Local-First" architecture, ensuring that automation logic runs on a local hub to maximize reliability and privacy. In contrast, Google Home remains largely "Cloud-Centric," leveraging its server-side processing power to deliver superior AI and voice assistant capabilities [6].

5.2. Setup Difficulty and User Experience

Google Home offers the most accessible entry point with a "Very Easy" setup process. Apple HomeKit follows a similarly streamlined "Easy" setup. SmartThings presents a "Medium" difficulty curve due to its extensive features. Conversely, Home Assistant is categorized as "Difficult" because it requires self-hosting and technical knowledge [16].

5.3. Legacy Protocol Support (Zigbee & Z-Wave)

Support for legacy local protocols is a key differentiator. SmartThings (via its Hub) and Home Assistant (via USB Dongles) provide native support for Zigbee and Z-Wave devices. Apple HomeKit and Google Home do not directly support these protocols; they require third-party bridges [7], [17].

5.4. Matter and Thread Support

Reflecting the industry's shift toward standardization, all four platforms—SmartThings, HomeKit, Google Home, and Home Assistant—have adopted the Matter standard and Thread networking. This ensures future-proof compatibility and reduces the reliance on proprietary bridges [9].

5.5. Data Privacy Models

Privacy approaches are strictly divided. Apple HomeKit and Home Assistant operate on a "No Data Collection" model, processing data locally to ensure maximum user privacy. On the other hand, Samsung SmartThings and Google Home operate on a service-based model where usage data is collected to personalize services and, in some cases, for advertising purposes [15], [18].

5.6. Offline Functionality

Reliability during internet outages is critical. Apple HomeKit and Home Assistant provide full offline functionality. SmartThings offers partial offline support through its Edge Drivers. Google Home has very limited offline capabilities [14].

6. Conclusion and Evaluation

This technical review of the four leading smart home platforms (SmartThings, HomeKit, Google Home, and Home Assistant) clearly reveals the fundamental design philosophies and architectural trade-offs to consider when developing a new smart home project. Instead of selecting the "best" platform, the primary lesson from this review is that each platform uses technology with a different strategy to meet a specific user need.

From a project development perspective, the key examples that can be taken from these systems are:

- **The Architectural Dilemma: Local vs. Cloud?**
    - **Apple HomeKit** and **Home Assistant** are the strongest examples of a **"local-first"** architecture. This approach offers critical advantages such as reliability (continuing to function during internet outages) and privacy (user data stays in-house). If reliability and privacy are priorities for your project, the "Home Hub" (Apple) or self-hosted (HA) models should be studied.
    - **Google Home** demonstrates the power of the **"cloud-centric"** approach. It leverages the cloud for advanced AI (Google Assistant), complex third-party integrations, and scalability. However, this comes with internet dependency and data privacy concerns.
    - **Samsung SmartThings** offers a pragmatic path by presenting a **"hybrid"** model between these two worlds. Its steps toward local control with "Edge Drivers" serve as a valuable example of how a balance can be struck between the flexibility of the cloud and the speed of local processing.
- **Interoperability Strategy:**
    - A project's success depends on how many devices it can communicate with. **SmartThings**' embrace of legacy protocols like Zigbee and Z-Wave has been key to gaining a large market share.
    - **Home Assistant** proves that the philosophy of "integrating everything" (3000+ integrations) is technically possible through the power of its open-source community.

    - **Apple's** strong support for the **Matter** standard is the most significant trend indicating the industry's evolution away from "walled gardens" and toward a common language. A new project must consider Matter as a foundational element for protocol support.

- ## User Experience and Data Privacy:
    - These platforms show that user experience (UX) is more than just an interface. For **Google Home**, the UX is *voice and AI*. For **Apple HomeKit**, the UX is *security, privacy, and ecosystem (Siri/iPhone) integrity*. For **Home Assistant**, the UX is *unlimited customization and control*.
    - Our project's approach to data privacy will be a fundamental architectural decision. **HomeKit's** end-to-end encryption and local analysis model (Secure Video) provides the best example of how to design a "privacy-first" product, while **Google** represents the value of data-driven personalization.

Requirements

This report outlines the architectural design and agile requirements for an IoT-Based Smart Home System. The project aims to provide a secure, efficient, and user-friendly environment by integrating environmental monitoring, automated resource management, and AI-supported security features. Before detailing the specific user stories, the high-level interaction between the system and external entities is illustrated below.

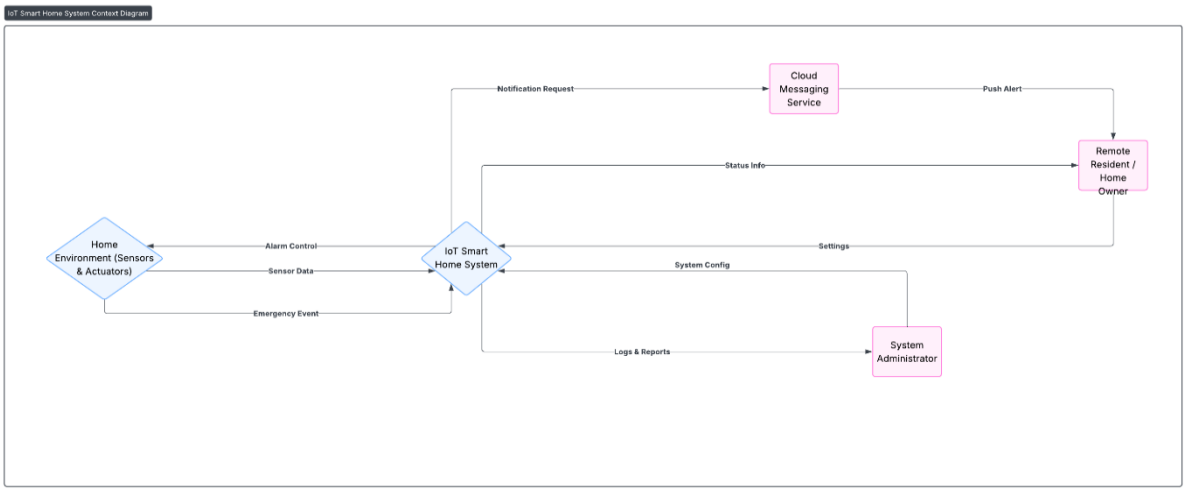

Figure 1 IoT Smart Home System Context Diagram

As illustrated in Figure 1, the system acts as a central hub connecting the Home Environment (Sensors/Actuators) with the Cloud Services and Users (Remote Residents/Admins). While local sensors feed data directly to the central unit, critical alerts and configurations are managed via secure cloud communication channels.

1. Agile Development Methodology and User Profiles

1.1 User Personas and Stakeholder Analysis

Key user profiles (personas) that will guide the system's architectural decisions have been defined in line with the project goals:

Persona 1: The Remote Resident

o **Definition**: An end-user concerned about home security and environmental conditions, familiar with technology but unwilling to deal with complex setups.

o **Expectation**: Prefers "peace of mind" over a passive data stream. Wants to be disturbed only in critical situations (fire, flood, burglary).

o **Interaction**: Receives instant notifications via the mobile app and accesses live camera feeds.

Persona 2: The Student/Admin (System Administrator)

o **Definition:** The technical expert developing the Smart Home System project and configuring the system.

o **Expectation**: Accessibility to low-level operations such as sensor calibration, Artificial Neural Network (ANN) model training, and database management.

o **Interaction:** Maintains the system via the Web-based Management Panel and hardware interfaces

2. Functional Requirements (Agile Epics and User Stories)

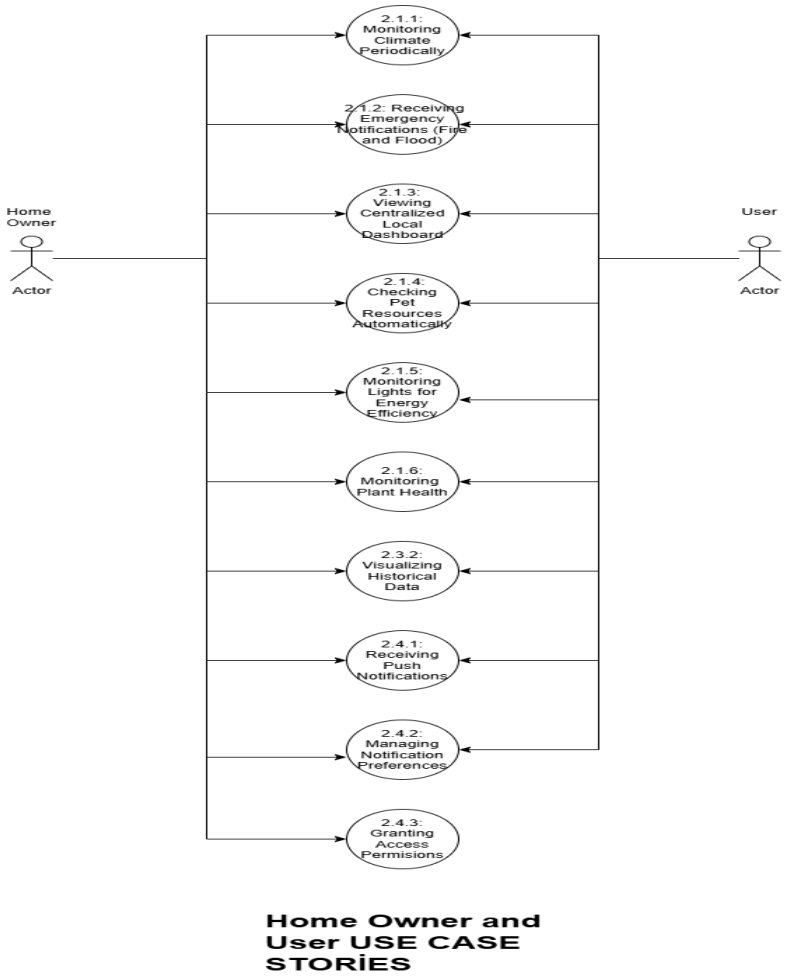

Figure 2 Comprehensive User Use Case Diagram and Requirement Map

The following diagram serves as a functional map of the system from the user's perspective. Each node in the diagram corresponds to a specific User Story detailed in the subsequent sections. Figure 2 details the interactions between the actors (Home Owner, User) and the system modules. The numbers assigned to each use case (e.g., 1.1, 1.2, 4.1) directly correspond to the subsection headers below, serving as a navigational guide.

2.1. Epic 1: Environmental Monitoring and Critical Hazard Management

This epic covers the system's fundamental IoT capabilities. The goal is continuous analysis of the home environment and immediate reporting of hazardous situations.

User Story 2.1.1: Tracking Climate Periodically

Story: As a resident, I want to see the temperature and humidity values of my home so that I can ensure a comfortable living space.

- **Technical Detail**: Sensor data is a continuous stream. However, sending data every second increases network traffic and database load. Therefore, a "polling" mechanism must be established.

- **Acceptance Criteria**:

o The system must read temperature and humidity sensors at a configurable interval (e.g., every 30 seconds).

o The read data must be packaged in JSON format with a timestamp and transmitted to the Cloud Server via HTTP POST or MQTT Publish.

o Web and Mobile interfaces must display the latest data with less than 1 minute of latency.

User Story 2.1.2: Detecting Emergency Hazards (Fire and Flood)

- **Story**: As a user, I want to be notified immediately of smoke or water leaks so I can take urgent action to protect my property.

- **Technical Detail**: Periodic reading is insufficient for emergencies. Therefore, an "Interrupt-based" architecture must be used.

- **Acceptance Criteria:**

o When the smoke or water sensor's digital output reaches the "HIGH" level, the system must interrupt the normal loop and trigger an "Emergency Event".

o This event must be transmitted to the server with a high-priority flag, and processing time must not exceed 5 seconds.

o A "Critical Alert" must be sent to the mobile application, designed to be noticeable even if the phone is on silent.

User Story 2.1.3: Displaying Centralized Local Dashboard

Story: As a resident, I want a dedicated, always-on wall display to view real-time sensor data and system status, allowing me to monitor the environment at a glance without needing to locate and unlock my mobile phone.

• **Technical Detail**: This involves integrating a touch-enabled LCD screen (e.g., 7-inch Touch Display) directly with the Raspberry Pi via DSI or HDMI. The software will run a lightweight local GUI optimized for touch interaction.

• **Acceptance Criteria:**

o The screen must provide a consolidated view of Temperature, Humidity, and Security Status (Armed/Disarmed).

o The interface must include a visual indicator (e.g., red blinking border) alongside the buzzer during critical events.

o Users must be able to silence active alarms or toggle security modes directly through the touch interface.

User Story 2.1.4: Monitoring Pet Resource Automatically

Story: As a pet owner, I want to track the weight of food and water bowls to receive alerts when they are running low, ensuring my pet's needs are met even when I am busy.

• **Technical Detail**: This requires integrating a Load Cell sensor with an HX711 amplifier module to the Raspberry Pi. The system must filter signal noise and convert the analog weight data into a digital percentage value.

• **Acceptance Criteria:**

o The system must be able to calibrate the empty and full weights of the containers.

o When the weight drops below a user-defined threshold (e.g., 10%), a "Low Food/Water" notification must be sent to the mobile app.

o To prevent notification fatigue, the alert should only be repeated after a specific cooldown period (e.g., 4 hours) if not refilled.

User Story 2.1.5: Monitoring Lights for Energy Efficiency

• **Story:** As an energy-conscious resident, I want to monitor the status of lights remotely and receive notifications if they are left on for an extended period, helping me reduce unnecessary electricity consumption.

• **Technical Detail:** This involves using LDR (Light Dependent Resistor) sensors or noninvasive current sensors (CT) to detect the state of lighting circuits. The backend must implement a timer logic that starts when a light is detected as "On" and triggers an alert if the duration exceeds a configured threshold.

• Acceptance Criteria:

o The mobile app must display the real-time status (On/Off) of monitored lights.

o The system must track the duration of the "On" state in real-time. o If a light remains active longer than a user-defined limit (e.g., 4 hours), an "Energy Waste Alert" notification must be pushed to the user.

User Story 2.1.6: Monitoring Plant Health

• **Story:** As a plant enthusiast, I want the system to monitor the soil moisture of my indoor plants and notify me when they need watering, preventing both dehydration and overwatering.

• **Technical Detail:** This utilizes Capacitive Soil Moisture Sensors connected via an Analog-to-Digital Converter (ADC) or digital GPIO pins. The system reads the voltage, maps it to a percentage (0-100%), and compares it against plant-specific thresholds.

• **Acceptance Criteria**:

o The dashboard must display the current moisture percentage of the monitored plant.

o A notification ("Time to Water: Living Room Fern") must be triggered when moisture drops below a critical level (e.g., 30%).

o To prevent false positives or spam, the alert should only be sent if the low moisture level persists for more than 10 minutes.

2.2. Epic 2: AI-Supported Security and System Automation

This section defines the system's internal automated tasks and "active security" features. The diagram below illustrates how the System acts as an autonomous actor to execute these tasks

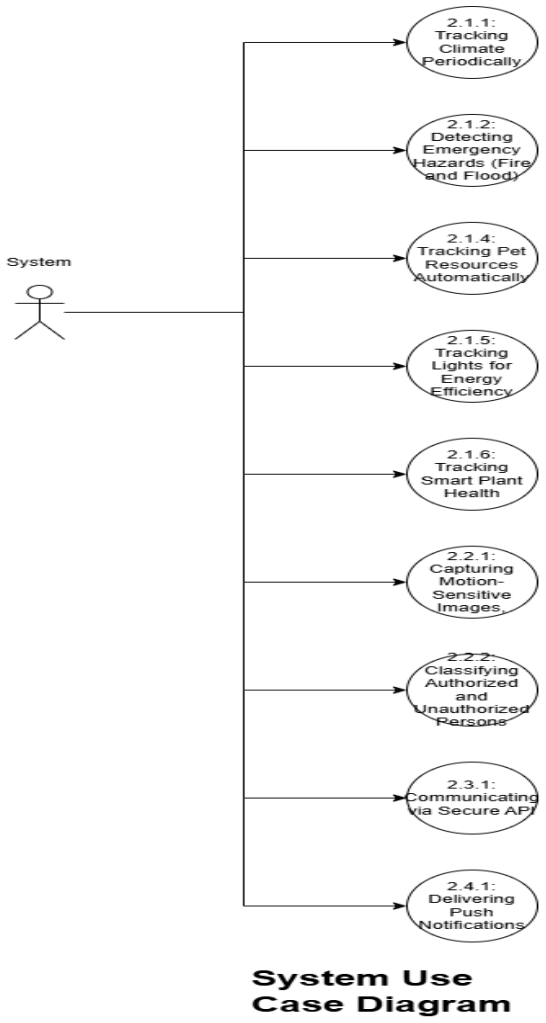

Figure 3 System Use Case Diagram.

Figure 3 highlights the backend processes initiated by the System itself, specifically focusing on Security (2.x), Communication (3.x), and automated tracking (1.x Series).

User Story 2.2.1: Capturing Motion-Sensitive Images

• **Story:** The system should activate the camera only when motion is detected, protecting my privacy and saving disk space.

• **Technical Detail:** While cloud-focused systems can stream continuously, local triggering is more efficient in terms of bandwidth and privacy.

• **Acceptance Criteria:**

o The signal from the PIR sensor must wake up the camera module.

o Within 500 milliseconds of triggering, the camera must capture a photo or record a short video (5-10 seconds).

o The captured image must be buffered in RAM for processing.

User Story 2.2.2: Classifying Authorized and Unauthorized Persons

• **Story:** As a resident, I want the system to distinguish between family members and strangers so I am not disturbed by false alarms.

• **Technical Detail:** Project goals require "identity detection" beyond motion. This necessitates the use of libraries like OpenCV or TensorFlow Lite.

• **Acceptance Criteria:**

o The captured image must be fed into a pre-trained Face Recognition or Human Detection model.

o The detected face must be compared with the registered "Residents" database.

o If the match rate is below a certain confidence threshold (e.g., 70%), the event must be labeled as "Unauthorized Intrusion".

o For authorized persons, only an "Entry Log" is kept, while for unauthorized persons, a security alarm is triggered.

2.3. Epic 3: Data Management and Cloud Backend

The cloud layer, the "Central Nervous System," ensures data persistence and accessibility as referenced in Use Case 3.1 and 3.2.

User Story 2.3.1: Communicating via Secure API

• **Story:** As a system admin, I want sensor data to reach the server via a secure channel.

• **Technical Detail:** Literature review shows that significant vulnerabilities in early smart home systems stemmed from unencrypted communication.

• **Acceptance Criteria:**

o The cloud application must offer RESTful API endpoints.

o All data traffic between the IoT module and the server must be encrypted using the HTTPS/TLS (Transport Layer Security) protocol.

o API requests must include API Key or Token-based authentication to prevent unauthorized data entry.

User Story 2.3.2: Viusalizing Historical Data

• **Story:** As a user, I want to view historical temperature changes in graphs.

• **Acceptance Criteria:**

o The database must store sensor readings using time-series logic.

o The web interface must allow the user to select a date range and visualize the data in that range as a line chart.

2.4. Epic 4: Remote Access and Mobile Experience

The interface layer ensures the user remains connected to their home at all times.

User Story 2.4.1: Delivering Push Notifications

• **Story:** As a mobile user, I want to be notified of critical events in my home even if the application is closed.

• **Technical Detail:** Constantly checking the app is impractical. Server-client interaction must be provided using services like cloud messaging applications.

Acceptance Criteria:

o When the server detects a critical event (Fire, Intruder), it must send a notification request to the CM service.

o The mobile app must receive this notification and provide audible and vibrating alerts to the user. o The notification content must clearly state the type and time of the event (e.g., "ATTENTION: Smoke Detected in Living Room - 14:30").

User Story 2.4.2: Managing Notification Preferences

• **Story:** As a Remote Resident, I want to be able to turn push notifications on or off at any time, so that I can avoid being disturbed when I am already at home or busy.

• **Technical Detail:** A boolean flag must be stored in the user's profile in the database. The backend service must check this flag before triggering the CM service.

• **Acceptance Criteria:**

o The mobile application must have a "Settings" tab with a toggle switch for notifications.

o Toggling the switch must send an API request to update the user's profile in the database.

o If the user disables notifications, the server must suppress the CM request even if a motion event is detected.

User Story 2.4.3: Granting Access Permissions

• **Story:** As a Home Owner, I want to register family members and trusted guests into the system database, granting them "authorized" status so that the security system recognizes them and does not trigger a false intruder alarm.

• **Technical Detail:** This functionality acts as the data entry point for the AI module defined in User Story 2.2.

• The mobile application must provide an interface to upload a face image or capture a new photo.

• This image is sent to the backend, where a face embedding (vector representation) is generated and appended to the allowed_faces database or serialized file (e.g., pickle). Acceptance Criteria:

o The mobile app must include a "Manage Residents" or "Add Person" screen. o The user must be able to assign a name/label to the captured face.

o Upon saving, the system must update the AI model's reference list immediately.

o The system must confirm that the new person is successfully recognized as "Authorized" in the next camera detection cycle.

3. Non-Functional Requirements (NFRs - Quality Attributes)

This section defines the system's operational standards, constraints, and quality attributes, ensuring the solution is not only functional but also robust, usable, and secure.

3.1. Security and Privacy

Security: As the system handles sensitive data within a private residence, security is paramount.

• **Data Encryption**: All data traffic between the IoT module, the Cloud Server, and the Mobile Application must be encrypted using HTTPS/TLS 1.2+ (Transport Layer Security).

• **Data Storage:** Sensitive user information (e.g., passwords, API keys) must be stored in the database using strong hashing algorithms (e.g., bcrypt or Argon2), never in plain text.

• **Visual Privacy:** To protect resident privacy, continuous video streaming must be disabled by default. Camera footage should only be processed locally on the Raspberry Pi, and only images related to a triggered security event shall be uploaded to the cloud.

3.2. Reliability and Availability

Resilience: The system must maintain core functionality even during infrastructure failures.

• **Offline Operation (Graceful Degradation):** In the event of an internet outage, the system must not fail completely. The Edge Node (Raspberry Pi) must continue to process sensor data locally, trigger the local buzzer for fire alarms, and buffer captured security images to the local SD card.

• **System Recovery**: In the event of a power failure, the IoT system must reboot and restore all monitoring services (sensors and API listeners) automatically within 90 seconds of power restoration.

• **Automatic Reconnection:** If the Wi-Fi connection is lost, the system must attempt to reconnect automatically every 30 seconds until connectivity is restored, without requiring manual user intervention.

3.3. Performance and Latency

Responsiveness: The system must react to critical physical events faster than human reaction time.

• Local Response Time: For life-critical events (Fire/Flood), the latency between the sensor detecting the hazard and the local alarm (buzzer) sounding must be under 100 milliseconds.
• Notification Latency: Assuming a stable internet connection (4G/Wi-Fi), the end-toend latency for a "Critical Alert" (from sensor trigger to the user receiving a Push Notification) must be under 5 seconds.

• AI Inference Speed: The face recognition algorithm running on the Raspberry Pi must process a captured image frame and return a classification result (Authorized/Unauthorized) within 3 seconds to ensure timely logging of intruders.

3.4. AI Accuracy and Precision

Intelligence: The automated decision-making modules must meet minimum accuracy thresholds to prevent false alarms.

• Classification Accuracy: The "Authorized/Unauthorized Person Classification" module must achieve a True Positive Rate (TPR) of at least 85% under normal indoor lighting conditions.

• False Positive Mitigation: The system must have a False Positive Rate (FPR) of less than 5% to prevent user "notification fatigue" caused by identifying family members as intruders.

• Sensor Precision: Temperature readings must be accurate within ±1°C, and humidity readings within ±5%, ensuring reliable environmental data for the user.

3.5. Usability and User Experience

Accessibility: The system is designed for the "Remote Resident" persona, who prioritizes ease of use over technical complexity.

• UI Responsiveness: The Mobile and Web interfaces must provide visual feedback (e.g., loading indicators, state changes) within 200 milliseconds of any user interaction (tap/click) to ensure a smooth experience.

• Learnability: A new user with no prior technical knowledge must be able to perform primary tasks (e.g., viewing the camera feed, silencing an alarm) within 2 minutes of first using the application.

• Error Handling: In case of a sensor failure or network error, the system must display user-friendly messages (e.g., "Living Room Sensor is offline") rather than technical error codes.

3.6. Scalability and Sustainability

Future-Proofing: The architecture must support future growth and comply with sustainability goals.

• Data Retention Policy: To manage storage on the Raspberry Pi's SD card, the system must implement a First-In-First-Out (FIFO) policy. When local storage usage exceeds 90%, the oldest non-critical logs and images must be automatically deleted.

• Modular Architecture: The software must adhere to Object-Oriented Programming (OOP) principles. Adding a new sensor type (e.g., a Light Sensor) should require creating a new class inheriting from the base "Sensor" class without modifying the core engine.

• Sustainability (SDG 9): The system infrastructure must be capable of logging energy consumption metrics (as part of the Extended Scope) to support future efficiency analysis features.

System Architecture

The purpose of this document is to outline the architectural design and system components of the proposed **IoT-Based Smart Home System**. This system aims to provide a secure, efficient, and user-friendly home monitoring environment by integrating environmental sensing, automated resource management, and AI-supported security features.

The design prioritizes an **"Edge-First"** approach. Critical triggers and initial processing are performed locally on the device to ensure reliability and privacy, while cloud services are leveraged for heavy processing (AI Face Matching), data persistence, and remote accessibility.

The high-level system architecture consists of the following key nodes:

- **Edge Layer (Raspberry Pi 5):**The Raspberry Pi 5 operates as the local edge processing unit, running the Sensor Driver Service and Rule Engine. It communicates with the cloud over Wi- Fi using secure protocols (HTTPS/MQTT). The system is designed to support *graceful degradation*, ensuring that critical local alarms remain functional even when internet connectivity is unavailable.
- **Cloud Backend:** The cloud backend hosts the REST API and database services. It is responsible for centralized state management, data persistence, and triggering push notifications via an external notification provider when critical events occur.
- **Client Applications:** The Web Dashboard and Mobile Application fetch data from the cloud node via API requests, allowing users to view real-time status and historical logs.

1. Design of Home Controller

The hardware architecture is centered around a **Edge Layer (Raspberry Pi 5)** single- board computer, acting as the main home controller and edge processing unit. The system integrates various sensors and actuators through specific digital and analog interfaces to minimize latency.

- **Main Controller (Raspberry Pi 5):** Selected for its high processing power, enabling local sensor management and preliminary event detection.
- **Vision Module:** A Raspberry Pi Camera Module connected via the **CSI (Camera Serial Interface)** port to provide high-bandwidth video data for the surveillance system.

- **Environmental Sensors:** The **DHT11** (Temperature & Humidity) and **MQ-2** (Gas & Smoke) sensors are connected via GPIO pins to facilitate climate tracking and emergency detection.
- **Resource Monitoring:** A Load Cell paired with an **HX711 Amplifier Module** connects to digital pins to precisely measure weight for the "Automated Pet Resource Monitoring" feature.
- **Storage:** A high-speed microSD Card (64GB) is utilized for the OS, local logs, and image buffering during offline operation (Graceful Degradation)

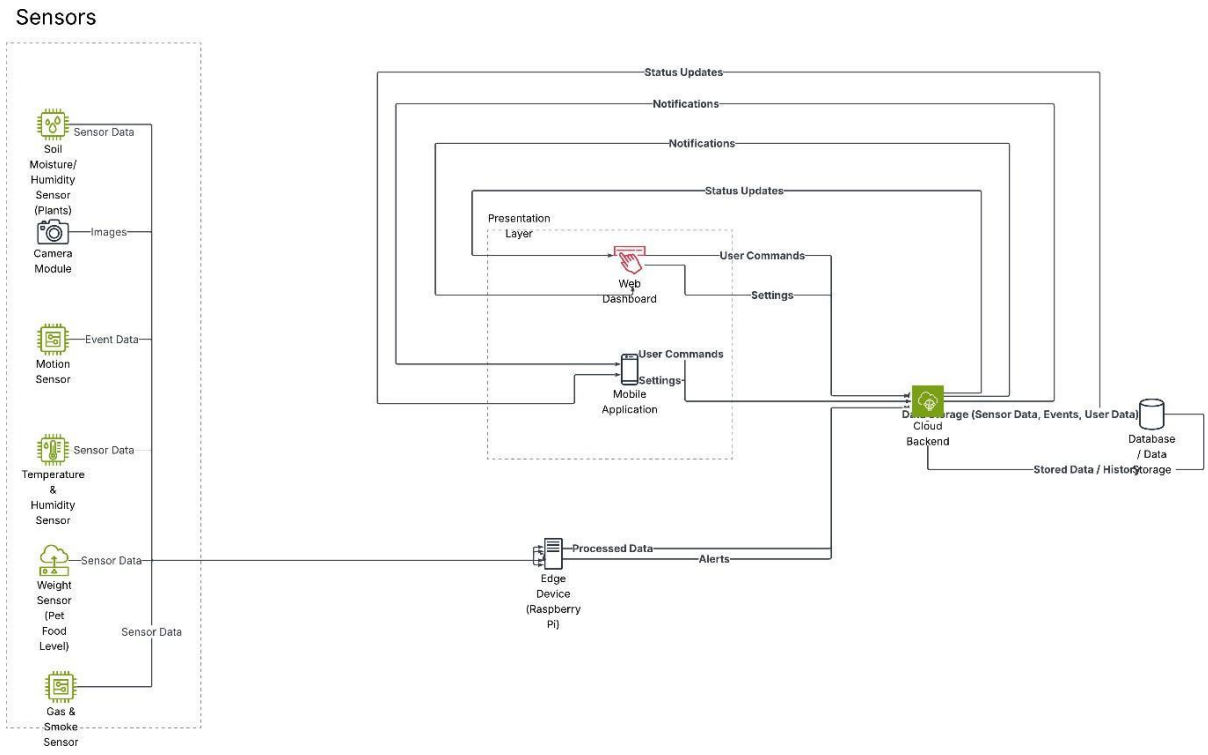

Figure 4 Design of Home Controller

2.Software Architecture Design

The software architecture follows a modular **4-Tier Layered Architecture** pattern to ensure separation of concerns, scalability, and maintainability.

2.1 Layer Decomposition

Layer 1: Presentation (User Interfaces)

This layer handles user interactions via the Mobile Application and Web Dashboard. It communicates solely through API calls, ensuring a stateless and responsive user experience. It allows users to view dashboards, receive notifications, and send manual commands (e.g., "Open Camera").

Figure 5 Mobile Application Security Notification

This figure illustrates a real-time security notification generated by the mobile application of the smart home system. When motion is detected and the captured individual cannot be identified as an authorized user, the system triggers an alert labeled *"Unknown Person Detected."* The notification provides a brief description of the event, enabling the user to take immediate action through the application.

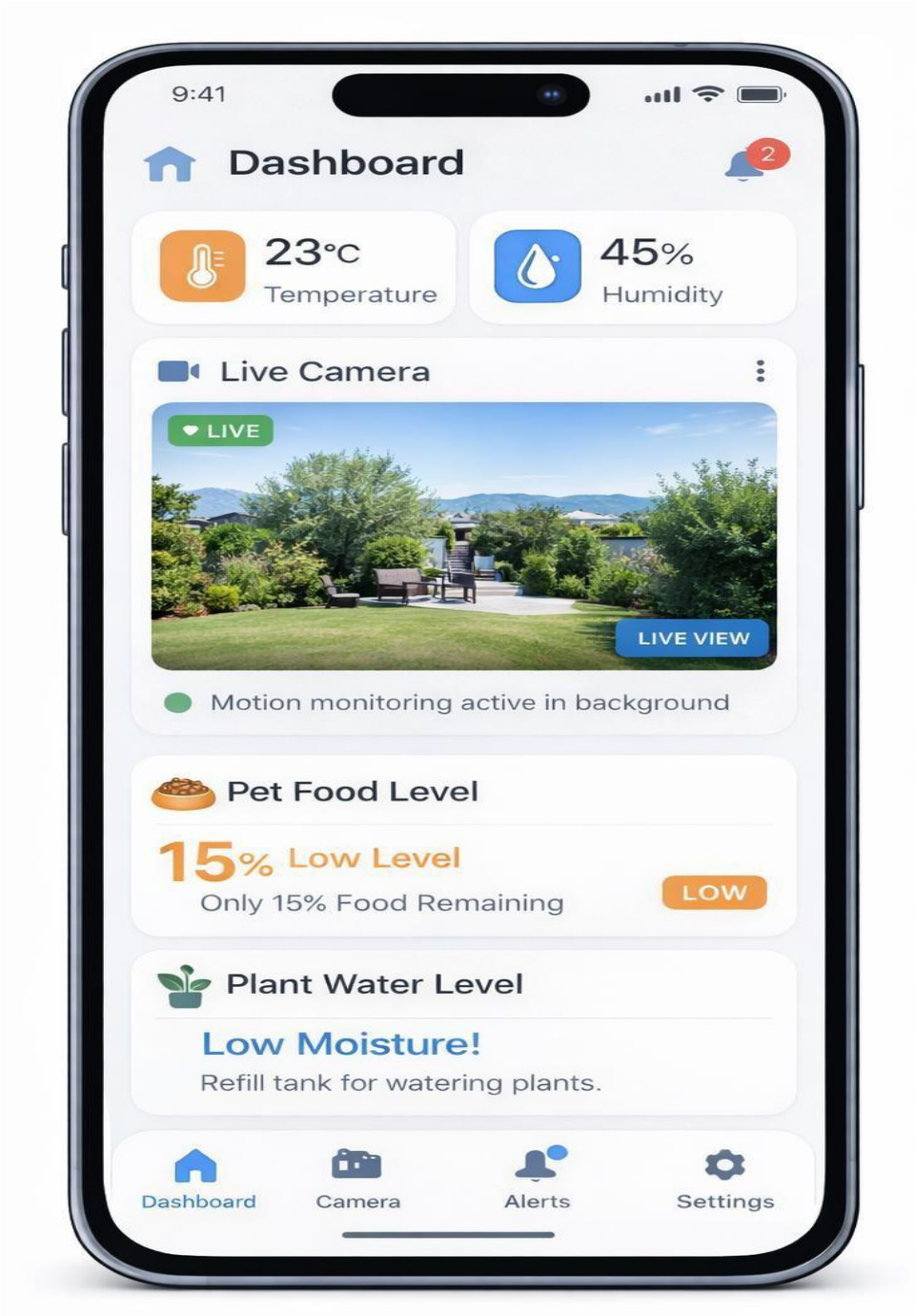

Figure 6 Mobile Application Dashboard Interface

This figure presents the main dashboard of the mobile application used in the smart home system. The interface displays real-time environmental data, including temperature and humidity levels, along with a live camera feed for continuous surveillance. Additionally, the dashboard provides status information for automated resource monitoring features such as pet food level and plant water level. Visual indicators and alerts are used to notify the user when critical thresholds are reached, enabling timely user intervention and efficient home management.

Figure 7 Web Dashboard Interface of the Smart Home System

This figure illustrates the web-based dashboard of the IoT smart home system. The interface provides a centralized view of real-time environmental data, including temperature and humidity levels, along with a live camera stream for continuous monitoring. In addition, the dashboard displays recent system alerts such as fire detection, intrusion events, and low resource warnings. Interactive controls and visual indicators enable users to acknowledge alerts, monitor system status, and manage home resources efficiently through a single web interface.

Layer 2: Security

This layer is responsible for user authentication during application login. Users are required to provide valid credentials (e.g., username and password) before accessing the system. Once authenticated, secure communication between the client applications and the backend is established using HTTPS/TLS. The security layer ensures that only authorized users can access system functionalities, while all business logic operations are executed after successful login verification.

Layer 3: Business Logic (Core Modules)

This is the core of the system where data processing occurs. To maintain a clean architecture and clear separation of duties, the functional requirements are grouped into four distinct handler modules:

1. **Climate Monitoring Service:** Handles periodic reading and logging of temperature and humidity data.
2. **Pet Resource Manager:** Manages the monitoring of food/water bowls and triggers alerts for low levels.
3. **Emergency Response Manager:** Dedicated to interrupt-based critical tasks like "Detecting Fire and Flood" to ensure immediate local feedback.
4. **Surveillance & Intrusion Analysis Engine:** A comprehensive security engine that handles the single-loop logic for motion capture, cloud-based face recognition, and the "Frequency Analysis" of unknown visitors.

Layer 4: Data & Integration

This layer manages persistence via the **Cloud Database** (storing user profiles, logs, and safe/non-safe face vectors) and handles third-party integrations, such as the External Push Notification Provider.

2. Detailed Module Design & Sequence Flows

This section details the internal algorithms, logic flows, and interaction sequences for the key functions defined in the Business Logic Layer.

2.1 Climate Monitoring Service

Addresses Requirements: `[Req. 2.1.1]`

Since continuous data streaming places unnecessary load on the network, this module utilizes a **"Polling Mechanism"**.

- **Logic:** The system wakes up at a configurable interval (default: 30s), reads the DHT11 sensor via GPIO, validates the data range, and transmits the packaged JSON payload to the Cloud Backend.
- **Sequence:** The interaction flow for periodic climate tracking is illustrated below.

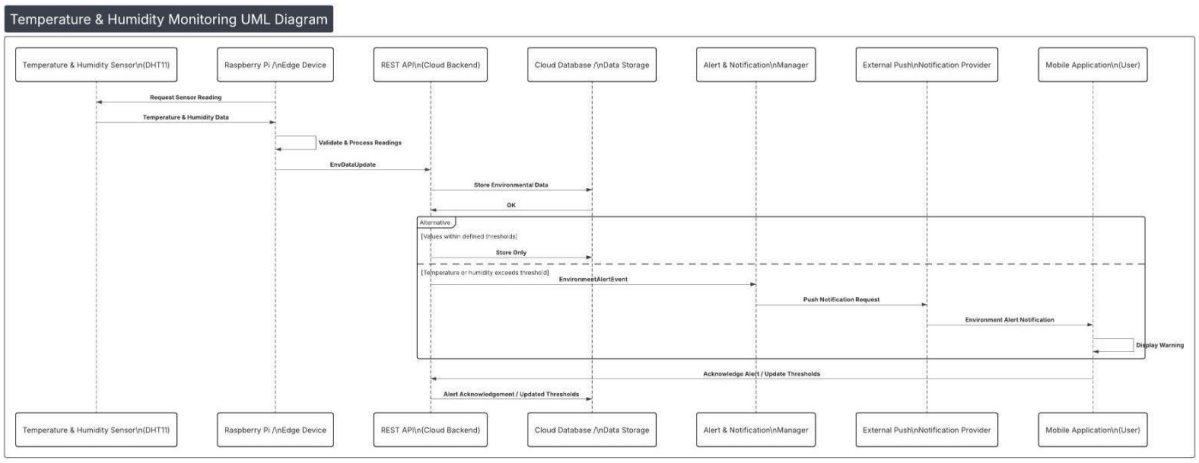

Figure 8 Temperature & Humidity Sequence Diagram

2.2 Emergency Response Manager

Addresses Requirements: `[Req. 2.1.2]` & `[Req. 3.2]`

Unlike the polling mechanism, emergency sensors operate on an **Interrupt-based Architecture** to ensure immediate response.

- **Logic:** When the Gas/Smoke sensor triggers, the main loop is interrupted. The Edge Node immediately triggers the local buzzer (Graceful Degradation) while simultaneously sending a high-priority alert to the Cloud Backend for push notifications.
- **Sequence:** The flow from detection to user notification is shown below.

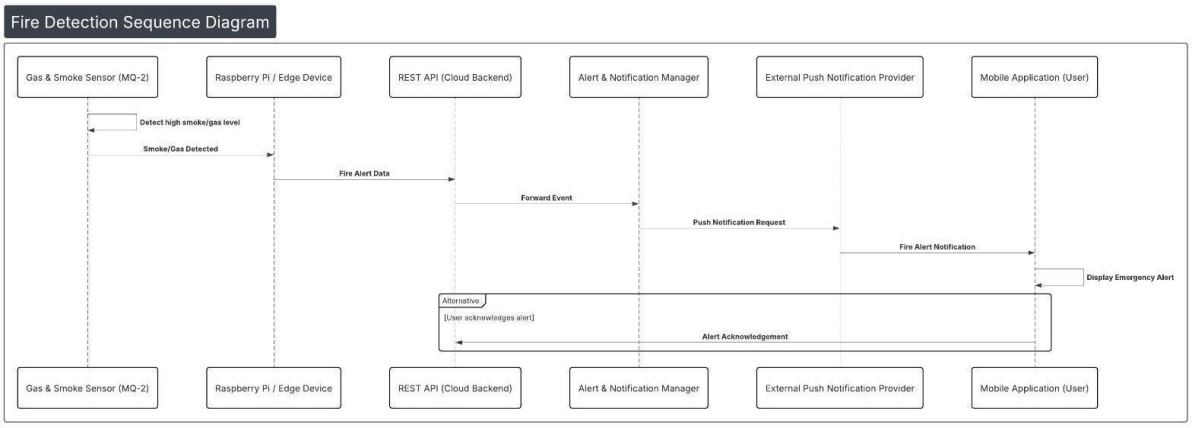

Figure 9 Fire Detection Sequence Diagram

2.3 Surveillance & Intrusion Analysis Engine

Addresses Requirements: `[Req. 2.2.1]`, `[Req. 2.2.2]`, `[Req. 2.4.3]`

The security system operates on a **Continuous Single-Loop Mechanism** to avoid complex multi-threading issues and ensure stability.

- Logic Flow:

1. **Sequential Check:** The system first checks for a "Manual Stream Request" from the app. If none, it checks the PIR Motion Sensor.
2. **Capture & Filter:** If motion is detected, images are captured. A local lightweight model checks for human presence.
3. **Cloud Identification:** Images are sent to the cloud. The AI compares the face against the "Safe User" database.
4. **Frequency Analysis:** If unauthorized, the system checks visit history (Frequency Analysis) and notifies the user with a visit count, asking for verification.

- **Sequence:** The complete flow from motion detection to AI classification and user notification is detailed below.

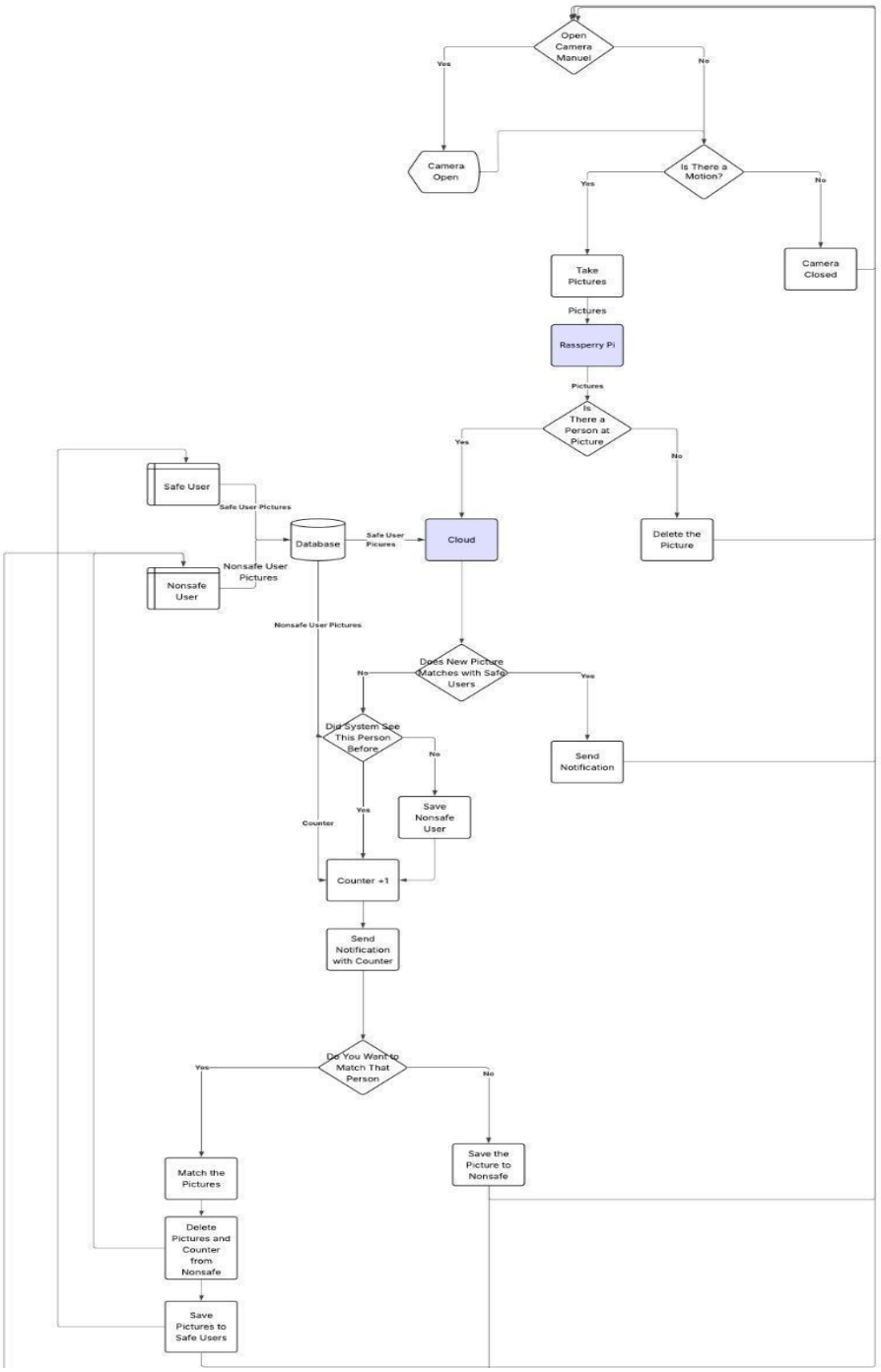

Figure 10 Classifying Authorized/Unauthorized Persons Data-flow Diagram

This figure illustrates the data flow and decision-making process of the surveillance and intrusion analysis module in the smart home system. The workflow begins with either a manual camera request from the user or motion detection by sensors. Captured images are initially processed on the edge device (Raspberry Pi) to detect human presence. If a person is identified, the images are forwarded to the cloud for face recognition by comparing them with registered safe user profiles stored in the database. When no match is found, the system performs a frequency analysis to determine whether the individual has been previously detected, increments a visit counter if necessary, and notifies the user accordingly. Based on user feedback, the detected individual may later be classified as a safe user, allowing the system to update stored profiles and improve future recognition accuracy.

3. Database Design

The database is designed to support user authentication, device management, sensor data storage, and event logging for the IoT-based smart home system. A centralized cloud database is used to ensure data persistence, scalability, and reliable access from both the mobile application and the web dashboard.

3.1 User Authentication Data

User-related data is stored to support secure application access. Basic user information and encrypted credentials are maintained in the database. Upon successful login, a session record is created to manage authenticated access. This structure ensures that only authorized users can interact with the system and access device data.

3.2 Device and Sensor Data

Each registered user may own one or more smart home devices. Devices are associated with multiple sensors, such as temperature, humidity, gas, motion, and load sensors. Sensor readings are stored as time-stamped records to enable real-time monitoring as well as historical data analysis through the client applications.

3.3 Event and Alert Management

System events, including emergency situations (e.g., fire detection), intrusion detection, and low resource levels, are recorded as alerts in the database. Alerts are linked to both the originating device and the corresponding user. This allows users to review recent alerts, acknowledge notifications, and track past incidents through the dashboard interfaces.

3.4 Surveillance Data

Camera-based security events are logged to support the surveillance and intrusion analysis features. Captured images and detection results are associated with camera events. Authorized user face profiles are stored separately and referenced during cloud-based face recognition processes to distinguish known users from unknown individuals.

3.5 Design Considerations

The database design follows a relational structure with clearly defined relationships between users, devices, sensors, and events. This approach ensures data integrity, simplifies access control after authentication, and supports future system extensions without requiring major architectural changes.

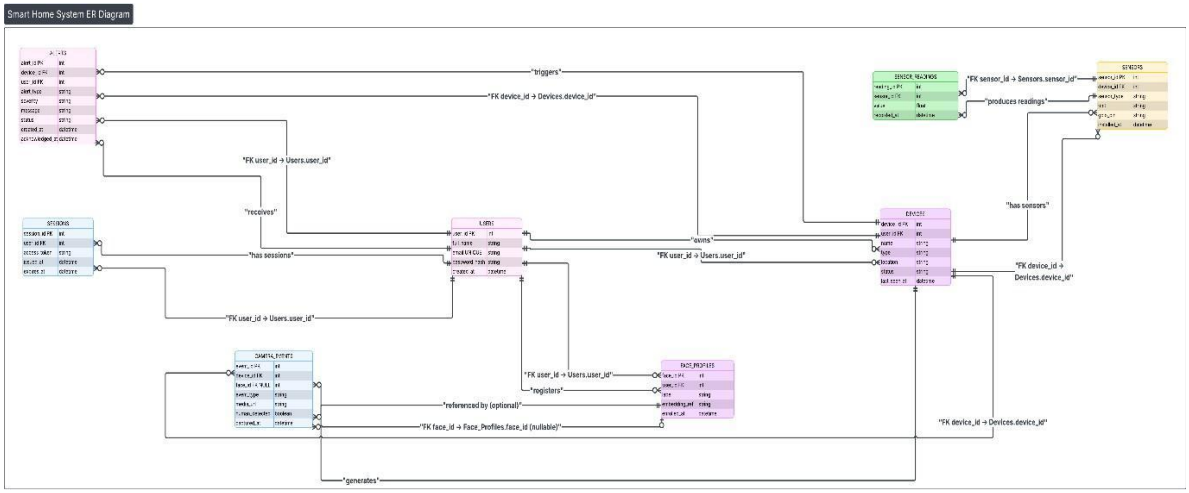

Figure 11 Entity-Relationship (ER) Diagram of the Smart Home System Database

This figure illustrates the Entity-Relationship (ER) diagram of the database designed for the IoT-based smart home system. The diagram presents the core entities, including users, devices, sensors, camera events, and alerts, along with their relationships and cardinalities. Primary and foreign keys are used to ensure data integrity, while optional relationships support scenarios such as unidentified individuals in camera-based surveillance. The database structure supports user authentication, real-time monitoring, event logging, and alert management in a centralized cloud environment.

4. Conclusion

This document presented the architectural design and system components of an IoT- based smart home system that integrates environmental monitoring, automated resource management, and AI-supported security features. By adopting an edge-first approach, the system ensures reliable local operation, reduced latency, and improved privacy while leveraging cloud services for data persistence, advanced processing, and remote accessibility.

The proposed layered architecture provides clear separation of concerns between user interfaces, security, business logic, and data management. Core functionalities such as climate monitoring, emergency response, resource tracking, and surveillance are designed as modular services, enabling scalability and ease of future extension. The surveillance and intrusion analysis workflow combines edge-based processing with cloud-based face recognition to enhance system responsiveness and security awareness.

The database design and data flow structures support secure user authentication, real- time monitoring, event logging, and alert management in a centralized manner. Overall, the proposed design offers a robust and flexible foundation for a smart home system and can be extended in future work to include additional sensors, automation rules, and advanced analytics.

IMPLEMENTATION

1. Introduction

The Internet of Things (IoT) has progressed beyond theoretical exploration to become a practical technology in everyday life, fundamentally transforming how people interact with their living spaces.

While commercial platforms such as Google Home, Amazon Alexa, and Apple HomeKit have popularized the concept of home automation with a focus on convenience and voice-based controls, a considerable gap remains between systems that offer simple, command-based automation and those that provide comprehensive environmental monitoring, intelligent security, and proactive automation.

This project, titled “A Smart Home System,” addresses these limitations by proposing a multi-faceted smart home solution integrating environmental monitoring, AI-enhanced security, and remote accessibility.

The system is built upon four interconnected components: IoT sensor modules deployed on a Raspberry Pi 5 edge controller, a FastAPI cloud gateway connected to a Supabase backend, a React-based web dashboard, and a Flutter mobile application.

The development process was divided into two academic phases. In CENG 407, the foundational architecture was designed, hardware components were selected and validated, and the initial sensor integration and basic backend were prototyped.

In CENG 408, the project was expanded into a fully functional system with complete cloud integration via Supabase, a production-ready face recognition pipeline, real-time dashboards on both web and mobile platforms, push notification delivery, and comprehensive resident management features.

This report details the implementation carried out in CENG 408, covering the developed features, the iterative development process, technical components, and system architecture decisions made throughout the development lifecycle.

2. System Overview

The Smart Home System follows a modular 4-Tier Layered Architecture designed for separation of concerns, scalability, and maintainability.

The system prioritizes an “Edge-First” approach: critical processing and sensor monitoring are performed locally on the Raspberry Pi to ensure reliability and privacy, while cloud services provide data persistence and remote accessibility.

The system architecture consists of the following core modules:

2.1 Edge Layer (IoT Sensor & AI Module)

The Raspberry Pi 5 serves as the central edge controller, running the run_edge.py orchestrator. This module manages all hardware interactions and local intelligence:

- Sensor Hardware: DHT11 (temperature & humidity), MQ-2 (gas/smoke detection), soil moisture sensor, and a PIR motion sensor are connected via GPIO pins. The Raspberry Pi Camera Module v2 is connected via the CSI interface.

• Sensor Reading Threads: A dedicated dht_reading_thread polls the DHT11 sensor every 2 seconds and transmits aggregated telemetry to the cloud at a configurable interval (default: 60 seconds). A digital_sensor_thread continuously monitors MQ-2, soil moisture, and PIR sensors at 1-second intervals with interrupt-like state change detection.

• Face Recognition Pipeline: Upon PIR motion detection, the camera captures a burst of images (configurable, default: 3 frames with 0.3-second delays). These frames are processed through a three-stage AI pipeline:

- Face Detection (FaceDetector): Uses MediaPipe Face Detection as the primary backend with face_recognition library as a fallback. Includes CLAHE-based image enhancement for low-light conditions.

- Embedding Extraction (FaceEmbedder): Generates 128-dimensional face embeddings using the face_recognition library. Supports both full-image and crop-based embedding extraction with padding for improved accuracy.

- Identity Matching (FaceMatcher): Compares extracted embeddings against locally cached resident profiles using Euclidean distance with a configurable threshold (default: 0.55). Classifies individuals as “authorized” or “unauthorized.”

• Cooldown Mechanism: A motion cooldown timer (default: 5 seconds) prevents excessive camera captures and redundant recognition cycles.

2.2 Cloud Backend (FastAPI Gateway & Supabase)

The cloud backend consists of two tightly integrated components:

FastAPI Gateway (main.py): A Python-based REST API server running on the same host as the edge controller also on a separate cloud server. It provides the following endpoints:

- POST /api/v1/telemetry/sensors — Receives sensor readings and writes them to the Supabase sensor_readings table.
- POST /api/v1/events/security — Creates security events in the events table.
- POST /api/v1/events/alert — Creates environmental alerts with configurable priority levels.
- POST /api/v1/events/upload-intelligent — Receives camera snapshots with face classification metadata and stores them in Supabase Storage.
- GET /api/v1/residents — Serves resident profiles with face embeddings to the edge device for local matching.
- POST /api/v1/heartbeat — Updates device online status and last-seen timestamp.
- POST /api/v1/residents/backfill-embeddings — Triggers on-demand embedding computation for resident photos.

Supabase Backend: A managed PostgreSQL database with real-time subscriptions, Row Level Security (RLS), and object storage. The database schema includes the following tables:

- profiles — Links Supabase Auth users to application profiles.
- devices — Registered IoT devices with online status, room assignment, and last-seen timestamps.
- sensor_readings — Time-series sensor data (temperature, humidity, smoke, water/soil).
- events — System events including alerts, security events, with acknowledgement tracking.
- camera_events — Camera-triggered events with snapshot paths and face detection metadata.
- event_faces — Individual face classification results per camera event, including match scores and bounding boxes.
- residents — Registered resident profiles with photo paths and JSONB face embeddings.

Resident Embedding Backfill Thread: A background daemon thread runs on the FastAPI gateway at configurable intervals (default: 45 seconds). It scans the residents table for entries with uploaded photos but missing embeddings, downloads photos from Supabase Storage, computes 128-dimensional face embeddings using the FaceEmbedder, and writes the resulting vectors back to the database.

2.3 Web Dashboard

The web dashboard is built with React 19 and Vite 7, providing a comprehensive management interface:

• **Technology Stack:** React, React Router v7, Supabase JS Client, Recharts for data visualization, Lucide React for iconography, and date-fns for date formatting.

• **Pages:** The application includes eight main pages:

- Dashboard: Real-time sensor readings, device status, and recent alerts overview.
- Rooms: Room-based device grouping with sensor data aggregation.
- History: Historical sensor data visualization with interactive line charts and date range filtering.
- Camera: Camera event gallery with snapshot images, face classification labels, and detection metadata.
- Alerts: Alert list with filtering, acknowledgement functionality, and priority indicators.
- Residents: Resident profile management including photo upload, face embedding status, and CRUD operations.
- Settings: User preferences and system configuration.
- Login: Supabase Auth-based email/password authentication.

• **Real-Time Updates:** The dashboard uses Supabase Realtime subscriptions to receive live updates for sensor readings, events, and camera events without polling.

• **Authentication & Authorization:** Protected routes with Supabase Auth integration. Row Level Security (RLS) policies ensure that only authenticated users can access data.

• **Theme Support**: Light and dark theme toggle with persistent preference storage.

2.4 Mobile Application

The mobile application is developed using Flutter (Dart SDK 3.9.2), targeting Android devices:

• State Management: Provider pattern with four main providers:
- AuthProvider: Manages Supabase authentication state and session lifecycle.
- SupabaseDataProvider: Centralized data layer for fetching and caching data.
- NotificationProvider: Real-time event subscriptions, and in-app alert popups.
- ThemeProvider: Handles light/dark theme switching with SharedPreferences persistence.

• Screens: The application includes six main screens accessible via a bottom navigation bar: Home (Dashboard), Camera, Alerts, Rooms, History, Settings, Residents, and Login.

• Push Notifications: Implemented using flutter_local_notifications for local notification display. The NotificationProvider subscribes to Supabase Realtime channels and generates local notifications for critical events even when the app is in the background.

• Real-Time Data: Supabase Realtime subscriptions for live sensor updates, new events, and camera event notifications.

• Offline Handling: Graceful error handling with user-friendly messages when network connectivity is unavailable.

2.5 Data Flow and Communication Architecture

The end-to-end data flow follows this path:

1. Sensor → Edge Controller: Hardware sensors transmit readings via GPIO pins to the Raspberry Pi run_edge.py process.
2. Edge Controller → FastAPI Gateway: Sensor telemetry, security events, alert events, and camera snapshots are sent via HTTP POST requests to the FastAPI gateway.
3. FastAPI Gateway → Supabase: The gateway validates incoming data, associates it with registered devices, and inserts records into the appropriate Supabase tables. Camera snapshots are uploaded to Supabase Storage.
4. Supabase → Client Applications: The web dashboard and mobile app receive real-time updates through Supabase Realtime subscriptions and fetch historical data via Supabase REST API queries.
5. Cloud → Edge (Resident Sync): The edge controller's ResidentSyncThread periodically polls the gateway's endpoint to download updated resident profiles with face embeddings.

The cloud module provides an additional resilience layer with:
- **Offline Queue:** SQLite-based request buffering for failed cloud uploads.
- **Sync Worker**: A background thread that drains the offline queue when connectivity is restored.
- **Connectivity Monitor:** Probes the backend health endpoint to track online/offline state transitions.
- **Heartbeat Sender:** Periodic health checks that keep the device's online status visible on dashboards.

3. Implemented Features

3.1 Environmental Monitoring System

The system continuously monitors the indoor environment using a suite of sensors:

• Temperature & Humidity: The DHT11 sensor is read in a dedicated background thread every 2 seconds. Valid readings are aggregated and transmitted to the cloud at the configured climate interval.
• Gas/Smoke Detection: The MQ-2 sensor operates on a state-change detection model. When the sensor's digital output transitions, the system immediately creates an alert event with appropriate severity.
• Soil Moisture Monitoring: The capacitive soil moisture sensor reports dry/wet binary state. State transitions trigger warning-level alerts for dry conditions and info-level notifications for moisture restoration.
Both web and mobile dashboards display these readings in real-time with automatic updates via Supabase.

3.2 Emergency Alert System

The emergency alert system uses an interrupt-based architecture for life-critical events:

• Local Response: Upon gas/smoke detection, the edge controller immediately logs the event and transmits a high-priority alert to the cloud backend.
• Cloud Processing: The FastAPI gateway creates a fire_alert event with critical priority in the Supabase events table.
• Push Notification Delivery: The mobile application subscribes to the Supabase Realtime channel. When a new critical event is detected, a local push notification is generated.
• Alert Acknowledgement: Users can acknowledge active alerts through both the web dashboard and mobile app. The acknowledged status is written back to the events table.

3.3 *AI-Based Face Recognition & Security Pipeline*

The face recognition system is the project's core security feature, operating as a complete pipeline from motion detection to identity classification:

- Motion Detection & Burst Capture: When the PIR sensor detects motion, the camera captures a burst of images.
- Face Detection: Each burst frame is processed by the FaceDetector class, which employs a dual-backend strategy (MediaPipe + face_recognition) and CLAHE-based image enhancement.
- Embedding Extraction: Detected face crops are passed to the FaceEmbedder, generating 128-dimensional vectors.
- Identity Matching: The FaceMatcher compares extracted embeddings against locally cached resident profiles. If the best match distance is ≤ 0.55, the person is classified as "authorized"; otherwise, "unauthorized".
- Best Frame Selection: The system processes all burst frames and selects the best result.
- Supabase Storage Upload: The selected frame with its classification results is uploaded to the FastAPI gateway, which creates a security event and stores the snapshot.
- Local Event Logging: All recognition events are logged to a local JSONL file for offline auditability.

3.4 Resident Management and Embedding Sync

The system provides a complete workflow for managing authorized residents:

- Adding Residents: Users can add new residents and optionally upload a face photo.
- Automatic Embedding Computation: The FastAPI gateway runs a background daemon thread that computes face embeddings for new photos and writes the vector back to the database.
- Edge Synchronization: The edge controller runs a ResidentSyncThread that fetches resident profiles with valid embeddings to update the local residents.json file.
- Deleting Residents: Resident deletion cascades properly, removing their embedding from the next sync cycle.

3.5 Real-Time Dashboard and Data Visualization

Both the web dashboard and mobile application provide comprehensive data visualization:

- Real-Time Sensor Display: Current sensor values are displayed with automatic real-time updates via Supabase Realtime channels.
- Historical Data Charts: Interactive line charts for historical sensor data, utilizing Recharts (web) and fl_chart (mobile).
- Camera Event Gallery: Camera events are displayed with snapshot thumbnails, face classification labels, match confidence scores, and timestamps.

3.6 Push Notification System

The notification system ensures users are informed of critical events:

- Mobile Implementation: Uses flutter_local_notifications to display push notifications for high-priority events (fire/gas detection, unauthorized person, low soil moisture).
- Notification Content: Includes the event type, a descriptive message, and a timestamp. Critical alerts are configured with maximum importance level.
- In-App Alert Popup: Tapping a notification navigates to the relevant screen.

3.7 Offline Resilience (Graceful Degradation)

The system is designed to maintain core functionality during internet outages:

- Local Sensor Processing: The edge controller continues to read sensors, trigger local alarms, and process face recognition entirely offline.
- Automatic Sync on Reconnection: A background drain cycle periodically checks backend connectivity and replays queued items when restored.
- Connectivity Monitoring: Probes the backend health endpoint to detect connectivity transitions.
- Local Event Log: All face recognition events are securely logged locally.

4. Implementation Details

4.1 Versioning

• v0.1 — Initial hardware setup: Raspberry Pi 5 configuration, GPIO pin assignments, sensor integration.
• v0.2 — Core sensor polling loop and basic cloud communication.
• v0.3 — Face recognition prototype with FaceMatcher.
• v0.4 — Cloud migration to Supabase, FastAPI gateway setup.
• v0.5 — Web dashboard implementation with real-time features.
• v0.6 — Mobile application completion and offline resilience layer.
• v1.0 — Final integrated system with end-to-end testing and UI refinements.

4.2 Iteration Log

• Iteration 1 (CENG 407): Hardware assembly and initial sensor integration.
• Iteration 2 (CENG 407–408 Transition): Core edge controller development (multi-threaded polling, motion detection).
• Iteration 3 (CENG 408): Cloud backend migration to Supabase and FastAPI.
• Iteration 4 (CENG 408): Web dashboard development with React/Vite.
• Iteration 5 (CENG 408): Mobile application development with Flutter.
• Iteration 6 (CENG 408): AI pipeline refinement (CLAHE, embedding backfill) and resilience module.
• Iteration 7 (CENG 408): Integration testing, performance tuning, and documentation.

4.3 Repository Reference

Repository: https://github.com/CankayaUniversity/ceng-407-408-2025-2026-A-Smarthome-System

Directory Structure:

• face-recognition/ — Edge controller, FastAPI gateway, and AI vision modules
• scripts/ — Test scripts
• cloud/ — Offline resilience module
• website/client/ — React/Vite web dashboard
• Mobile-app/ — Flutter mobile application
• supabase_setup.sql — Database schema and configuration

4.4 Screenshots

Screenshots are provided to demonstrate the implemented features and user interfaces of the Smart Home System.

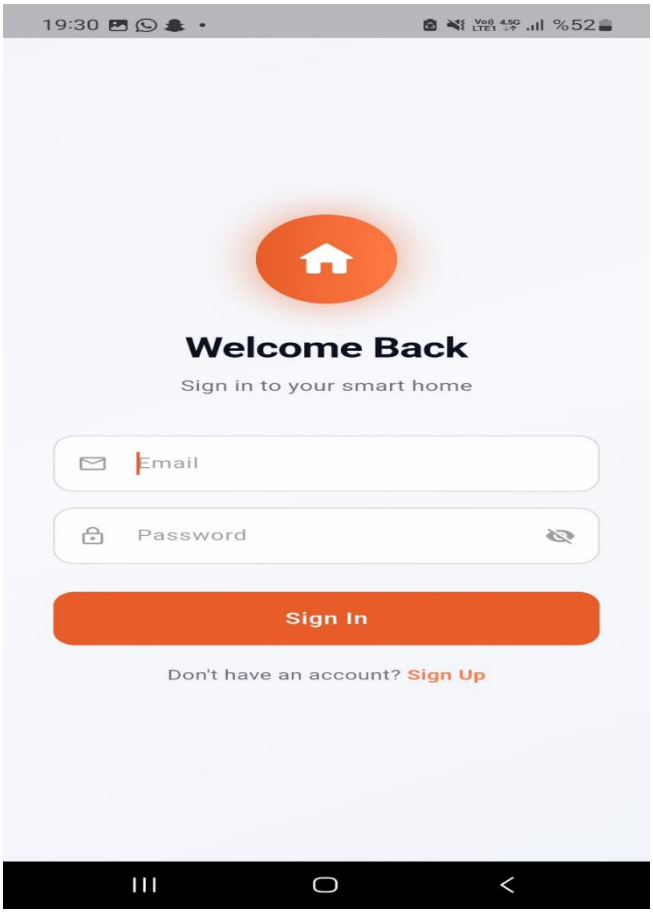

Figure 12 Mobile Application Login Screen

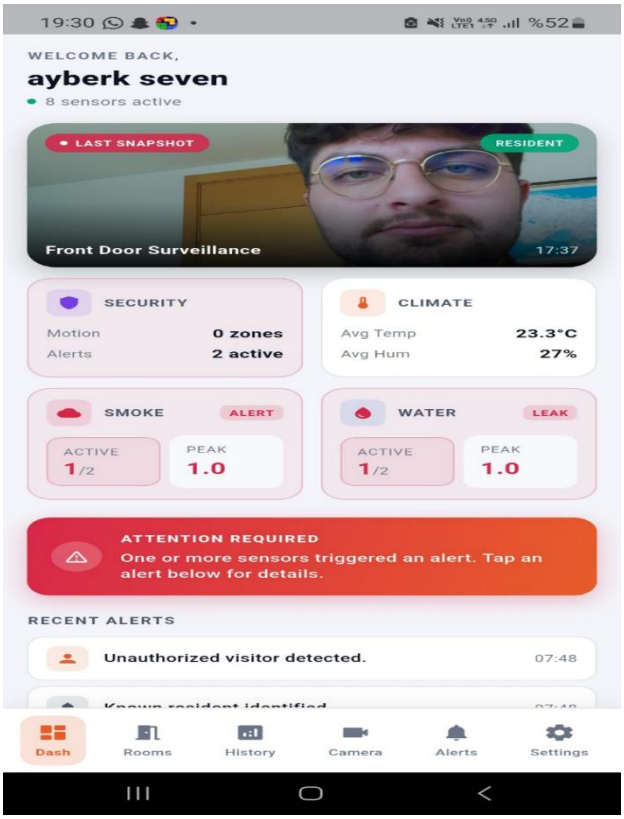

Figure 14 Mobile Application Dashboard — Real-Time Sensor

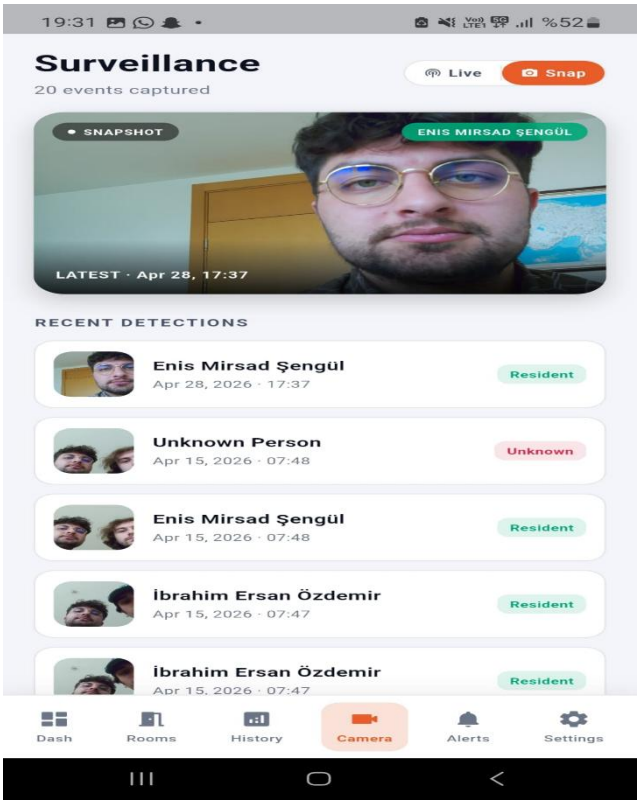

Figure 13 Mobile Application Camera Event — Face Recognition Result

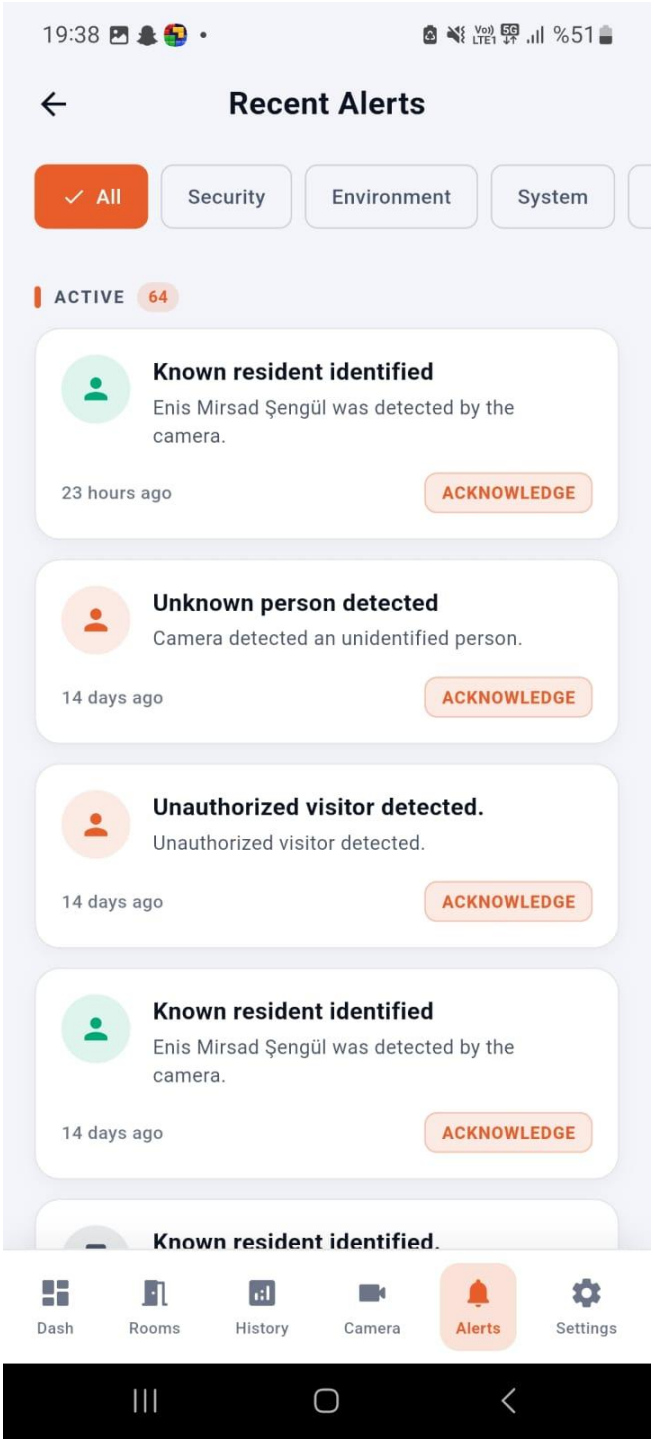

Figure 15 Mobile Application Alerts Screen — All Alerts

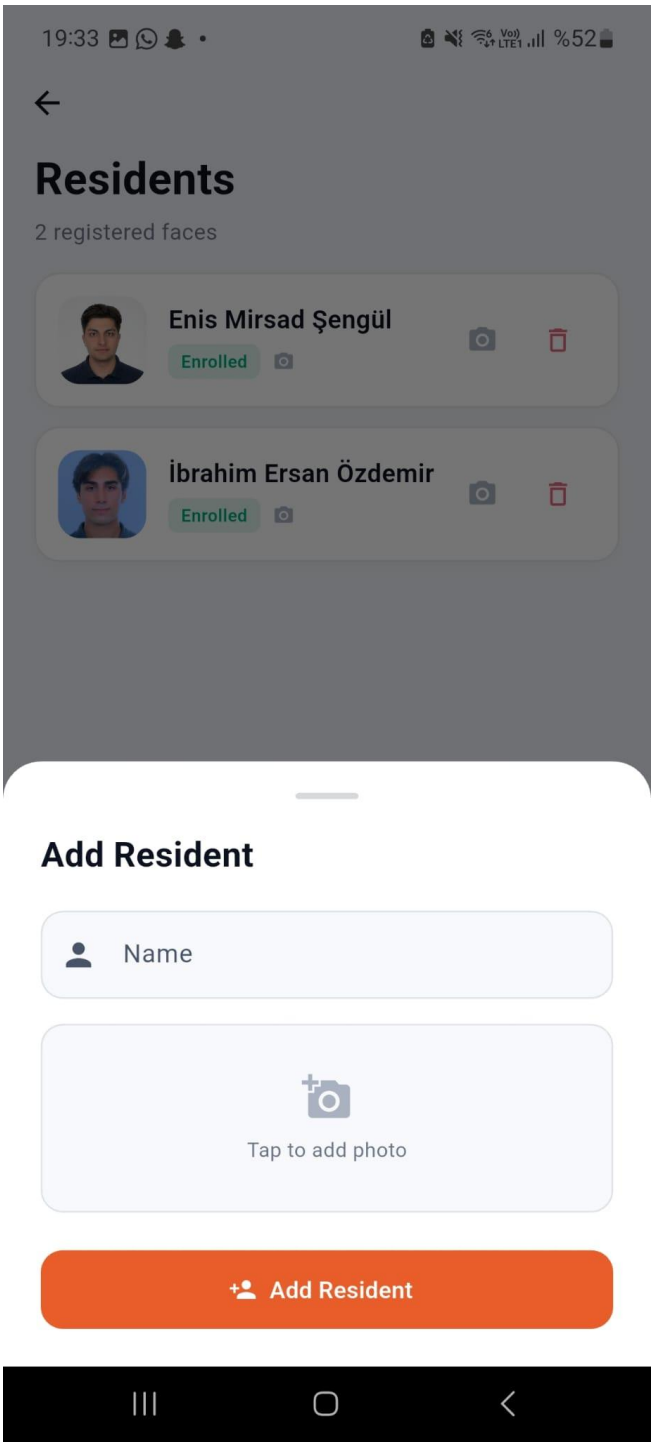

Figure 16 Mobile Application Residents Screen — Add New Resident

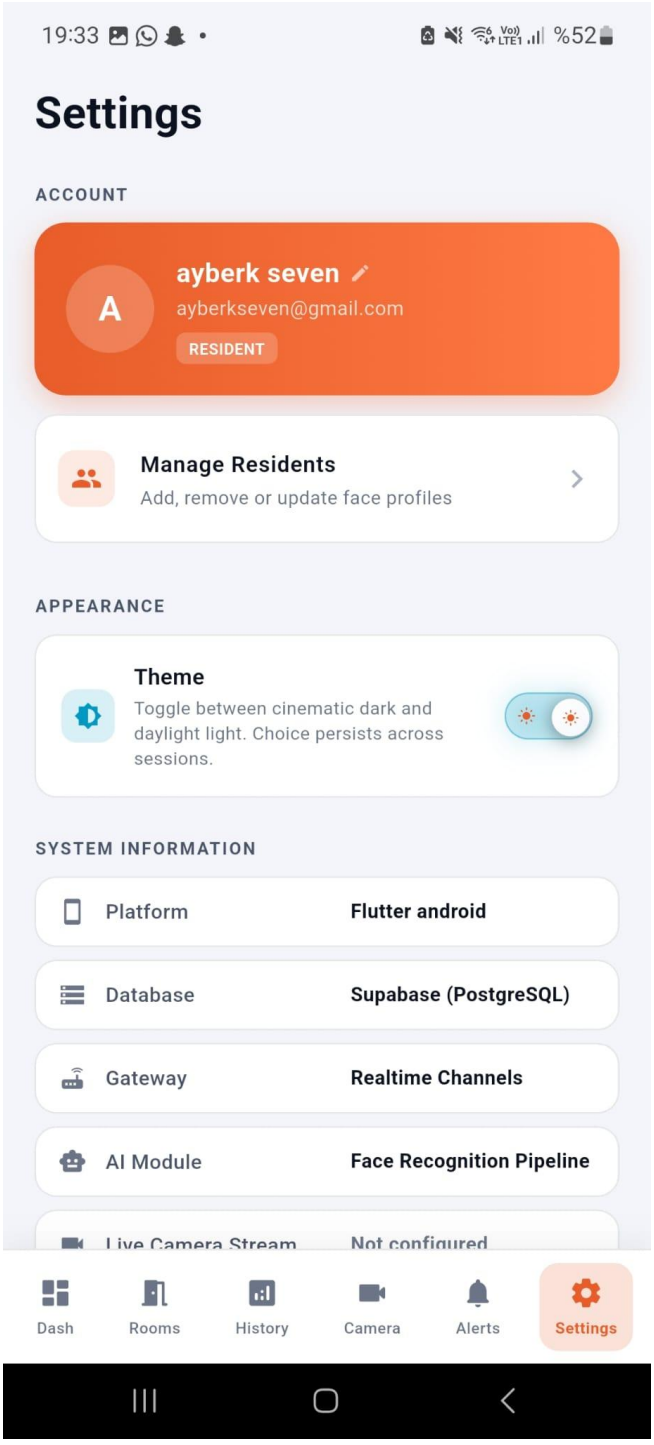

Figure 16 Mobile Application Settings Screen

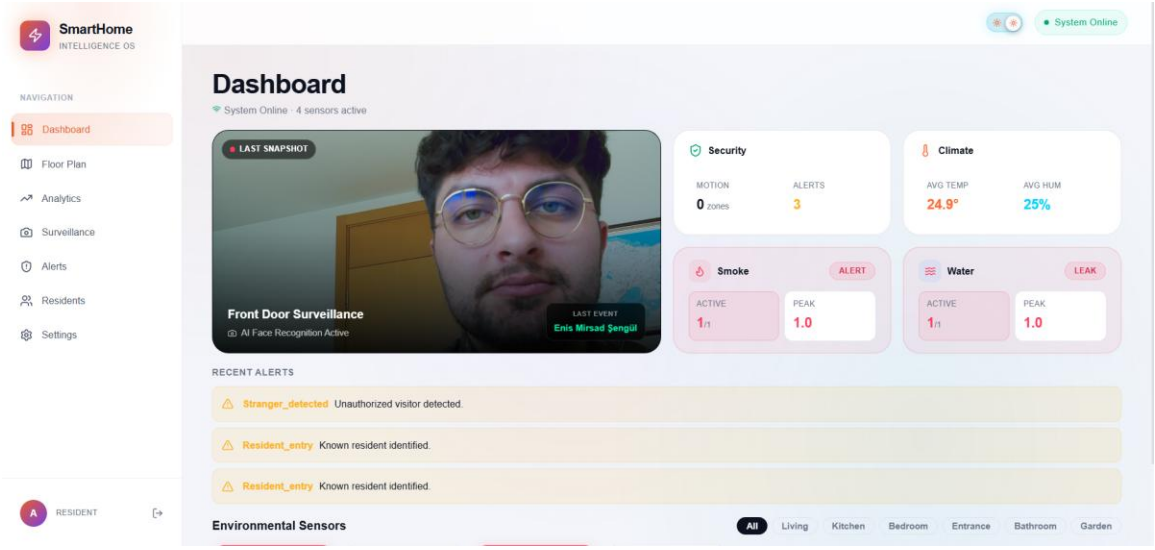

Figure 17 Web Dashboard — Main Dashboard with Sensor Cards and Recent Alerts

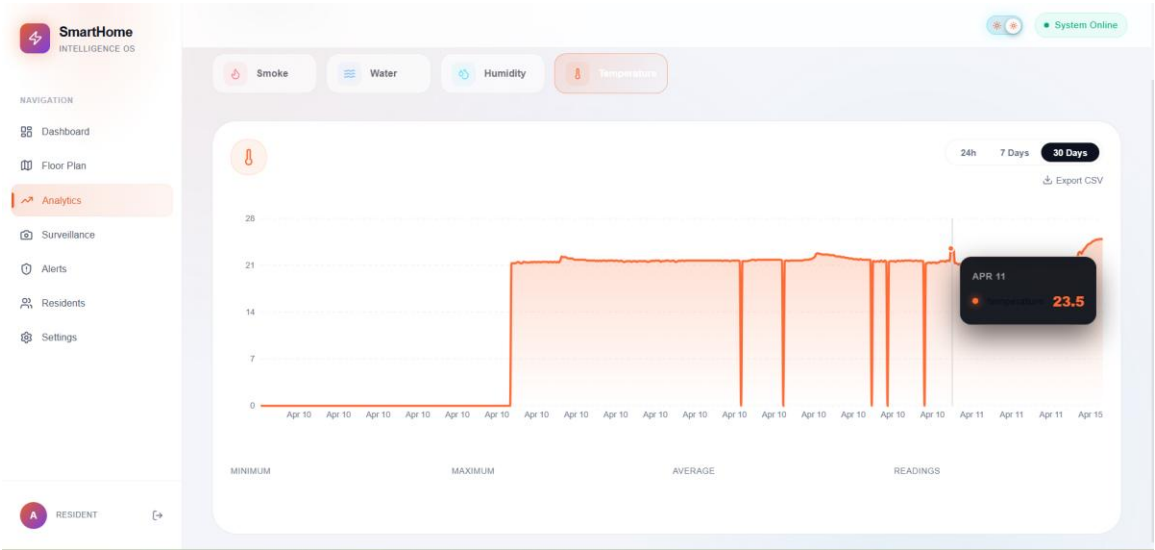

Figure 18 Web Dashboard — Historical Temperature and Humidity Charts

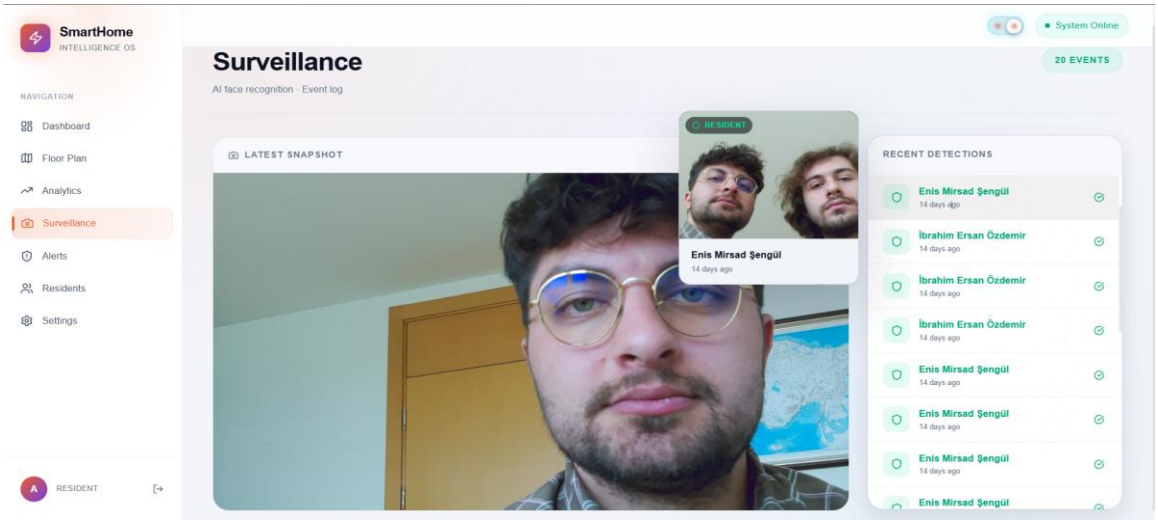

Figure 19 Web Dashboard — Camera Events with Face Classification

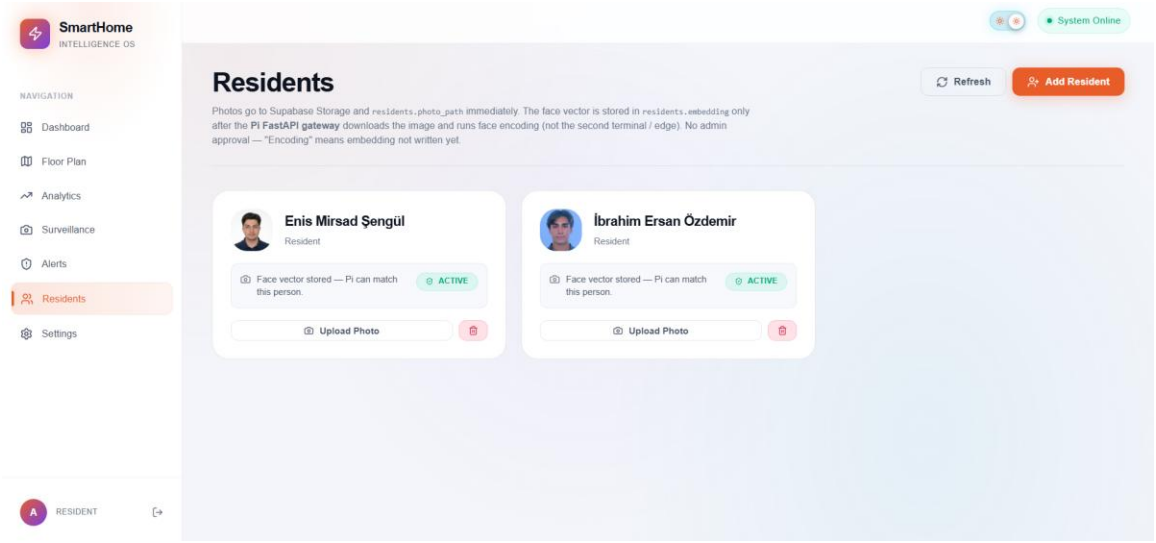

Figure 20 Web Dashboard — Resident Management with Embedding Status

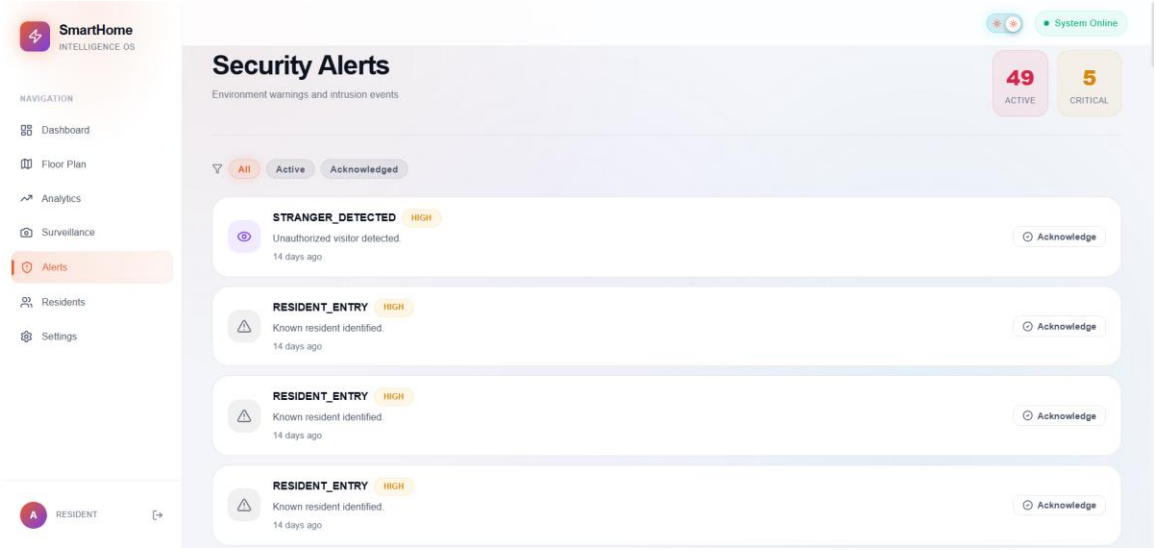

Figure 21 Web Dashboard — Alerts List with Acknowledgement

Testing & Validation

1.INTRODUCTION

1.1 Version Control

Version No	Description of Changes	Date
1.0	First version of the test plan document	April 29, 2026

1.2 Overview

This document describes the test plan, test design specifications, and test cases for the “A Smart Home System” project. The system integrates IoT sensor monitoring, AI-based face recognition for security, a cloud backend (Supabase), a Flutter mobile application, and a React-based web dashboard to provide comprehensive home automation, environmental monitoring, and intelligent security.

1.3 Scope

This document covers the functional testing of major features of the Smart Home System, including user authentication, environmental sensor monitoring (temperature, humidity, gas, soil moisture), emergency alert detection and notification, AI-based face recognition and intrusion detection, real-time dashboard visualization, and resident management. It defines pass/fail criteria and detailed test cases for all implemented modules across the mobile app, web dashboard, edge device (Raspberry Pi), and cloud backend.

1.4 Terminology

Acronym	Definition
UA	User Authentication
SM	Sensor Monitoring
EA	Emergency Alert System
FR	Face Recognition & Security
DV	Dashboard & Data Visualization
RM	Resident Management
IoT	Internet of Things
PIR	Passive Infrared (motion sensor)
DHT	Digital Humidity and Temperature sensor
MQ-2	Gas/Smoke sensor
API	Application Programming Interface
TC	Test Case

2.FEATURES TO BE TESTED

This section lists and gives a brief description of all the major features to be tested. For each major feature there will be a Test Design Specification added at the end of this document.

2.1 User Authentication (UA)

The system provides user authentication through Supabase. Users can sign in with email/password credentials via the mobile app and web dashboard. The system maintains session state and supports sign-out functionality.

2.2 Sensor Monitoring (SM)

The edge device (Raspberry Pi 5) reads environmental sensors (DHT11 for temperature/humidity, MQ-2 for gas/smoke, soil moisture sensor) at configurable intervals and transmits telemetry data to the cloud backend via a FastAPI gateway. Both web and mobile interfaces display real-time and historical sensor data.

2.3 Emergency Alert System (EA)

The system uses an interrupt-based architecture for critical events. When gas/smoke is detected or soil moisture drops below threshold, high-priority alerts are generated, stored in the database, and push notifications are sent to the mobile application.

2.4 Face Recognition & Security (FR)

Upon PIR motion detection, the camera captures burst images. A face detection pipeline (MediaPipe/face_recognition) identifies faces, generates embeddings, and compares them against registered resident profiles using Euclidean distance matching. Authorized persons are logged; unauthorized persons trigger security alerts with snapshot uploads.

2.5 Dashboard & Data Visualization (DV)

The web dashboard (React/Vite) and mobile app (Flutter) display real-time sensor readings,camera events, alerts, and device status. Historical sensor data is visualized through charts, and alerts can be acknowledged by users.

2.6 Resident Management (RM)

Users can add, update, and delete resident profiles through the mobile app and web dashboard. Resident photos are uploaded to Supabase Storage, face embeddings are automatically computed by a background thread, and the edge device periodically syncs resident data for local face matching.

3.FEATURES NOT TO BE TESTED

- **Scenario-Based Automation Engine:** The IFTTT-style rule engine for creating complex automation scenarios (e.g., “Away Mode”, “Night Mode”) is part of the extended scope and is not included in the current testing phase.
- **Energy Consumption Monitoring:** Integration of current transformer sensors for real-time power monitoring is planned as future work.
- **Local Touch Display Dashboard:** The Raspberry Pi touchscreen interface is under development and excluded from this testing cycle.
- **Pet Resource Monitoring (Load Cell):** While the hardware is integrated, the weight-based pet food/water monitoring feature is not fully calibrated for testing.

4.ITEM PASS/FAIL CRITERIA

When the actual output of the system matches the expected output described in the test case, the test case is considered passed. When there is a discrepancy between expected and actual behavior, including incorrect data display, failed API responses, missed notifications, or incorrect face classification, the test case fails.

4.1 Exit Criteria

Testing is considered successful when:

- 100% of the test cases are executed
- At least 90% of the test cases pass
- All High priority test cases pass
- No critical functional errors remain in the core modules (authentication, sensor monitoring, emergency alerts, face recognition)

5.TEST DESIGN SPECIFICATIONS

5.1 User Authentication (UA)

5.1.1 Subfeatures to be tested

- **Email/Password Login (UA.EP):** This subfeature allows users to sign in using their registered email address and password via the Supabase authentication service on both the mobile app and web dashboard.
- **Sign Out (UA.SO):** This subfeature allows authenticated users to sign out of the application, clearing the session and redirecting to the login screen.

5.1.2 Test Cases

TC ID	Requirements	Priority	Scenario Description
UA.EP.01	2.4	H	Enter valid email and password and verify successful login
UA.EP.02	2.4	H	Enter valid email and blank password and verify error
UA.EP.03	2.4	M	Enter invalid email format and verify error message
UA.SO.01	2.4	H	Sign out from the app and verify redirect to login screen

5.2 Sensor Monitoring (SM)

5.2.1 Subfeatures to be tested

- **Climate Telemetry (SM.CT):** This subfeature reads temperature and humidity data from the DHT11 sensor at configurable intervals and transmits readings to the cloud via the FastAPI gateway.
- **Gas Detection (SM.GD):** This subfeature monitors the MQ-2 sensor for gas/smoke detection using GPIO digital input and reports state changes to the backend.
- **Soil Moisture (SM.SM):** This subfeature monitors the soil moisture sensor and reports dry/wet state transitions to the backend.

5.2.2 Test Cases

TC ID	Requirements	Priority	Scenario Description
SM.CT.01	2.1.1	H	Verify temperature and humidity readings are sent at configured interval
SM.CT.02	2.1.1	H	Verify sensor readings appear on the dashboard within 1 minute
SM.GD.01	2.1.2	H	Trigger gas sensor and verify state change is reported
SM.GD.02	2.1.2	M	Clear gas condition and verify “cleared” alert is sent
SM.SM.01	2.1.6	H	Verify dry soil state triggers a low moisture alert

SM.SM.02	2.1.6	M	Verify soil moisture restored notification after re-watering

5.3 Emergency Alert System (EA)

5.3.1 Subfeatures to be tested

- **Fire/Gas Alert (EA.FA):** This subfeature generates critical alerts when the MQ-2 sensor detects gas/smoke, triggers the local buzzer, and sends push notifications.
- **Alert Notification Delivery (EA.ND):** This subfeature delivers push notifications to the mobile app when critical events occur, even if the app is in the background.
- **Alert Acknowledgement (EA.AK):** This subfeature allows users to acknowledge alerts through the dashboard or mobile app.

5.3.2 Test Cases

TC ID	Requirements	Priority	Scenario Description
EA.FA.01	2.1.2	H	Trigger gas sensor and verify critical alert is created in database
EA.FA.02	2.1.2	H	Verify alert notification latency is under 5 seconds
EA.ND.01	2.4.1	H	Verify push notification is received on mobile for fire alert

EA.ND.02	2.4.1	M	Verify notification content includes event type and timestamp
EA.AK.01	2.4	H	Acknowledge an alert and verify status update in database

5.4 Face Recognition & Security (FR)

5.4.1 Subfeatures to be tested

- **Motion Detection & Capture (FR.MC):** This subfeature activates the camera when the PIR sensor detects motion and captures a burst of images for analysis.
- **Face Detection (FR.FD):** This subfeature processes captured images using MediaPipe or face_recognition library to detect face bounding boxes.
- **Face Embedding & Matching (FR.FM):** This subfeature generates 128-dimensional face embeddings and compares them against registered resident embeddings using Euclidean distance with a configurable threshold.
- **Security Event Upload (FR.SU):** This subfeature uploads the best frame with face classification results to the cloud and creates security events.

5.4.2 Test Cases

TC ID	Requirements	Priority	Scenario Description
FR.MC.01	2.2.1	H	Trigger PIR sensor and verify camera captures burst images
FR.MC.02	2.2.1	M	Verify motion cooldown prevents
FR.FD.01	2.2.2	H	Present a face to camera and verify face is detected

FR.FD.02	2.2.2	M	Verify no-face frames are correctly skipped
FR.FM.01	2.2.2	H	Present a registered resident and verify authorized classification
FR.FM.02	2.2.2	H	Present an unregistered person and verify unauthorized classification
FR.FM.03	2.2.2	M	Verify match score is below threshold (0.55) for authorized persons
FR.SU.01	2.2.2	H	Verify security event with snapshot is uploaded to Supabase
FR.SU.02	2.2.2	H	Verify unauthorized detection triggers an intrusion alert

5.5 Dashboard & Data Visualization (DV)

5.5.1 Subfeatures to be tested

Real-time Sensor Display (DV.RS): This subfeature displays current temperature, humidity, gas status, and soil moisture on both the web dashboard and mobile app.

Historical Data Charts (DV.HC): This subfeature visualizes historical sensor readings as charts with date range selection.

Camera Event Display (DV.CE): This subfeature shows recent camera events with face detection results, snapshot images, and classification labels.

5.5.2 Test Cases

TC ID	Requirements	Priority	Scenario Description
DV.RS.01	2.3.2	H	Verify real-time sensor values are displayed on the dashboard
DV.RS.02	2.3.2	M	Verify sensor values update when new readings arrive
DV.HC.01	2.3.2	H	Verify historical temperature chart renders correctly
DV.HC.02	2.3.2	M	Select a date range and verify chart data filters accordingly
DV.CE.01	2.2	H	Verify latest camera event with snapshot is displayed
DV.CE.02	2.2	M	Verify face

5.6 Resident Management (RM)

5.6.1 Subfeatures to be tested

Add Resident (RM.AR): This subfeature allows users to add a new resident with a name and optional photo through the mobile app or web dashboard.

Delete Resident (RM.DR): This subfeature allows users to remove a resident from the system.

Embedding Sync (RM.ES): This subfeature automatically computes face embeddings for newly uploaded resident photos and syncs them to the edge device.

5.6.2 Test Cases

TC ID	Requirements	Priority	Scenario Description
RM.AR.01	2.4.3	H	Add a new resident with name and photo and verify database entry
RM.AR.02	2.4.3	M	Add a resident without a photo and verify entry is created
RM.DR.01	2.4.3	H	Delete a resident and verify removal from database
RM.ES.01	2.4.3	H	Upload resident photo and verify embedding is computed automatically

TC ID	Requirements	Priority	Scenario Description
RM.ES.02	2.4.3	H	Verify edge device syncs updated resident embeddings

6.DETAILED TEST CASES

6.1 UA.EP.01

TC_ID	UA.EP.01
Purpose	Verify that a user can log in with valid email and password.
Requirements	2.4
Priority	High
Estimated Time Needed	3 Minutes
Dependency	A user account must be registered in Supabase.
Setup	The application is running and the login screen is displayed.
Procedure	[A01] Open the mobile app or web dashboard. [A02] Enter a valid registered email address. [A03] Enter the correct password for this account. [A04] Tap/click the “Sign In” button. [V01] Observe that the login is successful and the dashboard screen appears.
Cleanup	Sign out.

6.2 UA.EP.02

TC_ID	UA.EP.02
Purpose	Verify that login fails when password is blank.
Requirements	2.4
Priority	High
Estimated Time Needed	2 Minutes
Dependency	None.
Setup	The login screen is displayed.
Procedure	[A01] Enter a valid email address. [A02] Leave the password field blank. [A03] Tap/click the “Sign In” button. [V01] Observe that an error message is displayed. [V02] Verify that the user remains on the login screen.
Cleanup	None.

6.3 UA.EP.03

TC_ID	UA.EP.03
Purpose	Verify that an invalid email format is rejected.
Requirements	2.4
Priority	Medium

Estimated Time Needed	2 Minutes
Dependency	None.
Setup	The login screen is displayed.
Procedure	[A01] Enter an invalid email format (e.g., “abc@”). [A02] Enter any password. [A03] Tap/click the “Sign In” button. [V01] Verify that a validation error message is displayed.
Cleanup	None.

6.4 UA.SO.01

TC_ID	UA.SO.01
Purpose	Verify that the user can sign out and is redirected to the login screen.
Requirements	2.4
Priority	High
Estimated Time Needed	2 Minutes
Dependency	UA.EP.01 should pass.
Setup	The user is logged in and on the dashboard.
Procedure	[A01] Navigate to the Settings screen. [A02] Tap/click the “Sign Out” button. [V01] Observe that the user is redirected to the login screen. [V02] Verify that the session is cleared.
Cleanup	None.

6.5 SM.CT.01

TC_ID	SM.CT.01
Purpose	Verify that temperature and humidity readings are transmitted at the configured interval.
Requirements	2.1.1
Priority	High
Estimated Time Needed	5 Minutes
Dependency	Edge device must be running with DHT11 sensor connected.
Setup	The Raspberry Pi edge controller is powered on and the FastAPI gateway is running.
Procedure	[A01] Start the edge controller (run_edge.py). [A02] Wait for two climate intervals (default 60s each). [V01] Check the Supabase sensor_readings table for new entries. [V02] Verify that temperature and humidity readings are recorded with timestamps. [V03] Verify that the interval between readings matches the configured CLIMATE_INTERVAL_SECONDS.
Cleanup	None.

6.6 SM.CT.02

TC_ID	SM.CT.02
Purpose	Verify that sensor readings appear on the dashboard within 1 minute.
Requirements	2.1.1
Priority	High
Estimated Time Needed	3 Minutes
Dependency	SM.CT.01 should pass.
Setup	Edge device is running and the dashboard is open.
Procedure	[A01] Open the web dashboard or mobile app dashboard. [A02] Wait for a new sensor reading cycle. [V01] Verify that the displayed temperature and humidity values update. [V02] Verify the latency is less than 1 minute.
Cleanup	None.

6.7 SM.GD.01

TC_ID	SM.GD.01
Purpose	Verify that gas/smoke detection triggers a state change report.
Requirements	2.1.2
Priority	High
Estimated Time Needed	3 Minutes

Dependency	Edge device must be running with MQ-2 sensor connected.
Setup	The edge controller is running and gas sensor reads “NO GAS”.
Procedure	[A01] Simulate gas detection by triggering the MQ-2 sensor. [V01] Verify that the edge controller logs “Gas: ALERT”. [V02] Verify that a fire_alert event is created in the events table. [V03] Verify that the alert severity is “critical”.
Cleanup	Clear the gas condition.

6.8 EA.FA.01

TC_ID	EA.FA.01
Purpose	Verify that a critical alert is created in the database when gas is detected.
Requirements	2.1.2
Priority	High
Estimated Time Needed	3 Minutes
Dependency	SM.GD.01 should pass.
Setup	Edge controller is running in normal state.

Procedure	[A01] Trigger the MQ-2 gas sensor. [V01] Query the Supabase events table. [V02] Verify a new event with event_type “fire_alert” and priority “critical” exists. [V03] Verify the event message contains “Gas/smoke detected”.
Cleanup	Clear gas condition.

6.9 EA.ND.01

TC_ID	EA.ND.01
Purpose	Verify that a push notification is received on the mobile device for a fire alert.
Requirements	2.4.1
Priority	High
Estimated Time Needed	5 Minutes
Dependency	EA.FA.01 should pass. The mobile app must be installed with notifications enabled.
Setup	Mobile app is installed and user is logged in. App may be in background.
Procedure	[A01] Trigger a gas detection event on the edge device. [V01] Observe the mobile device for a push notification. [V02] Verify the notification appears with audible/vibration alert. [V03] Verify the notification content states the event type.
Cleanup	Dismiss the notification.

6.10 EA.AK.01

TC_ID	EA.AK.01
Purpose	Verify that acknowledging an alert updates its status in the database.
Requirements	2.4
Priority	High
Estimated Time Needed	3 Minutes
Dependency	EA.FA.01 should pass.
Setup	An unacknowledged alert exists in the system.
Procedure	[A01] Open the Alerts screen on the mobile app or web dashboard. [A02] Tap/click the acknowledge button on an active alert. [V01] Verify the alert status changes to “acknowledged” in the UI. [V02] Query the events table and verify the acknowledged field is true.
Cleanup	None.

6.11 FR.MC.01

TC_ID	FR.MC.01
Purpose	Verify that the camera captures burst images when motion is detected.
Requirements	2.2.1
Priority	High

Estimated Time Needed	5 Minutes
Dependency	Edge device with PIR sensor and camera module connected.
Setup	The edge controller is running and no motion is active.
Procedure	[A01] Trigger the PIR motion sensor by moving in front of it. [V01] Observe the edge controller log for “DETECTED” motion state. [V02] Verify that burst images are captured (check data/ directory). [V03] Verify the number of captured images matches BURST_COUNT configuration.
Cleanup	Remove captured test images.

6.12 FR.FD.01

TC_ID	FR.FD.01
Purpose	Verify that the face detection pipeline identifies faces in captured images.
Requirements	2.2.2
Priority	High
Estimated Time Needed	5 Minutes
Dependency	FR.MC.01 should pass.
Setup	Edge controller is running with camera active.
Procedure	[A01] Stand in front of the camera and trigger motion. [V01] Observe edge logs for face detection results. [V02] Verify that the log reports the number of faces detected. [V03] Verify that bounding box coordinates are logged for each face.
Cleanup	None.

6.13 FR.FM.01

TC_ID	FR.FM.01
Purpose	Verify that a registered resident is classified as “authorized”.
Requirements	2.2.2
Priority	High
Estimated Time Needed	5 Minutes
Dependency	FR.FD.01 should pass. RM.ES.01 should pass.
Setup	A resident is registered with a valid face embedding in the system.
Procedure	[A01] The registered resident stands in front of the camera. [A02] Trigger motion detection. [V01] Observe the edge logs for face matching results. [V02] Verify the status is “authorized”. [V03] Verify the recognized name matches the resident’s name. [V04] Verify the match score (Euclidean distance) is below the threshold (0.55).
Cleanup	None.

6 .14 FR.FM.02

TC_ID	FR.FM.02
Purpose	Verify that an unregistered person is classified as “unauthorized”.
Requirements	2.2.2
Priority	High
Estimated Time Needed	5 Minutes
Dependency	FR.FD.01 should pass.
Setup	The person in front of the camera is NOT registered in the residents database.
Procedure	[A01] An unregistered person stands in front of the camera. [A02] Trigger motion detection. [V01] Observe the edge logs for face matching results. [V02] Verify the status is “unauthorized”. [V03] Verify that a “stranger_detected” security event is created.
Cleanup	None.

6.15 FR.SU.01

TC_ID	FR.SU.01
Purpose	Verify that a security event with snapshot is uploaded to Supabase.
Requirements	2.2.2
Priority	High

Estimated Time Needed	5 Minutes
Dependency	FR.FM.01 or FR.FM.02 should pass.
Setup	The edge device is online and connected to the FastAPI gateway.
Procedure	[A01] Trigger a face recognition event (motion + face detected). [V01] Query the Supabase events table for a new security event. [V02] Query the camera_events table for a snapshot_path entry. [V03] Verify the image exists in the Supabase Storage bucket. [V04] Verify the event_faces table contains the classification result.
Cleanup	None.

6.16 DV.RS.01

TC_ID	DV.RS.01
Purpose	Verify that real-time sensor values are displayed on the dashboard.
Requirements	2.3.2
Priority	High
Estimated Time Needed	3 Minutes
Dependency	SM.CT.01 should pass. User must be authenticated.
Setup	The edge device is sending telemetry and the user is logged in.
Procedure	[A01] Open the dashboard (web or mobile). [V01] Observe the temperature reading widget. [V02] Observe the humidity reading widget. [V03] Verify that the displayed values are within expected ranges. [V04]

	Verify that gas and soil moisture statuses are displayed.
Cleanup	None.

6.17 DV.HC.01

TC_ID	DV.HC.01
Purpose	Verify that historical sensor data is rendered as a chart.
Requirements	2.3.2
Priority	High
Estimated Time Needed	3 Minutes
Dependency	Multiple sensor readings must exist in the database.
Setup	The user is logged in and navigates to the history page.
Procedure	[A01] Navigate to the History screen. [V01] Verify that a line chart is displayed for temperature data. [V02] Verify that data points correspond to stored sensor readings.
Cleanup	None.

6.18 DV.CE.01

TC_ID	DV.CE.01

Purpose	Verify that the latest camera event with snapshot is displayed.
Requirements	2.2
Priority	High
Estimated Time Needed	3 Minutes
Dependency	FR.SU.01 should pass.
Setup	A camera event with snapshot exists in the database.
Procedure	[A01] Open the Camera screen on the mobile app or web dashboard. [V01] Verify that the latest camera event is listed. [V02] Verify that the snapshot image is loaded and visible. [V03] Verify that the face classification label is displayed (e.g., “Resident” or “Unknown”).
Cleanup	None.

6.19 RM.AR.01

TC_ID	RM.AR.01
Purpose	Verify that a new resident can be added with name and photo.
Requirements	2.4.3
Priority	High
Estimated Time Needed	5 Minutes
Dependency	User must be authenticated.

Setup	The user is on the Residents screen.
Procedure	[A01] Navigate to the Residents screen. [A02] Tap/click “Add Resident”. [A03] Enter the resident’s name. [A04] Upload a face photo. [A05] Confirm the addition. [V01] Verify that the new resident appears in the residents list. [V02] Query the Supabase residents table and confirm the entry exists. [V03] Verify that the photo is uploaded to Supabase Storage.
Cleanup	Delete the test resident if needed.

6.20 RM.DR.01

TC_ID	RM.DR.01
Purpose	Verify that a resident can be deleted from the system.
Requirements	2.4.3
Priority	High
Estimated Time Needed	3 Minutes
Dependency	RM.AR.01 should pass.
Setup	A test resident exists in the system.
Procedure	[A01] Navigate to the Residents screen. [A02] Select the test resident. [A03] Tap/click the delete option. [A04] Confirm the deletion. [V01] Verify the resident is removed from the UI list. [V02] Query the Supabase residents table and confirm the entry is deleted.

Cleanup	None.

6.21 RM.ES.01

TC_ID	RM.ES.01
Purpose	Verify that face embedding is automatically computed for uploaded resident photos.
Requirements	2.4.3
Priority	High
Estimated Time Needed	5 Minutes
Dependency	RM.AR.01 should pass. FastAPI gateway must be running.
Setup	A resident with a photo but no embedding exists.
Procedure	[A01] Add a new resident with a valid face photo. [A02] Wait for the resident embedding backfill thread to run. [V01] Query the Supabase residents table for the new resident. [V02] Verify that the embedding field is populated with a 128-dimensional vector.
Cleanup	None.

6.22 RM.ES.02

TC_ID	RM.ES.02
Purpose	Verify that the edge device syncs updated resident embeddings from the cloud.
Requirements	2.4.3
Priority	High
Estimated Time Needed	5 Minutes
Dependency	RM.ES.01 should pass. Edge device must be running.
Setup	A resident with a computed embedding exists in Supabase.
Procedure	[A01] Ensure the edge controller is running with resident sync thread active. [A02] Wait for the sync interval to elapse (default 60s). [V01] Check the local residents.json file on the Raspberry Pi. [V02] Verify that the newly embedded resident appears in the local file. [V03] Verify that the resident's embedding data matches the cloud data.
Cleanup	None.

6.23 SM.GD.02

TC_ID	SM.GD.02
Purpose	Verify that a “cleared” alert is sent when the gas condition ends.
Requirements	2.1.2
Priority	Medium
Estimated Time Needed	3 Minutes
Dependency	SM.GD.01 should pass.
Setup	The edge controller is running and gas sensor currently reads “ALERT”.
Procedure	[A01] Remove the gas source from the MQ-2 sensor. [A02] Wait for the sensor state to transition to “NO GAS”. [V01] Verify the edge controller logs the state change to “NO GAS”. [V02] Query the Supabase events table for a “fire_alert_cleared” event. [V03] Verify the alert severity is “info”.
Cleanup	None.

6.24 SM.SM.01

TC_ID	SM.SM.01
Purpose	Verify that dry soil state triggers a low moisture alert.
Requirements	2.1.6
Priority	High

Estimated Time Needed	3 Minutes
Dependency	Edge device must be running with soil moisture sensor connected.
Setup	The edge controller is running and soil sensor reads “WET”.
Procedure	[A01] Remove the sensor from moist soil to simulate a dry condition. [V01] Verify the edge controller logs “Soil: DRY”. [V02] Query the Supabase events table for a “low_moisture” event. [V03] Verify the alert severity is “warning” and the message contains “critically low”.
Cleanup	Restore the sensor to moist soil.

6.25SM.SM.02

TC_ID	SM.SM.02
Purpose	Verify that a “moisture restored” notification is sent after re-watering.
Requirements	2.1.6
Priority	Medium
Estimated Time Needed	3 Minutes
Dependency	SM.SM.01 should pass.
Setup	The edge controller is running and soil sensor currently reads “DRY”.

Procedure	[A01] Place the sensor back into moist soil. [A02] Wait for the sensor state to transition to “WET”. [V01] Verify the edge controller logs “Soil: WET”. [V02] Query the Supabase events table for a “moisture_restored” event. [V03] Verify the alert severity is “info”.
Cleanup	None.

6.26 EA.FA.02

TC_ID	EA.FA.02
Purpose	Verify that the alert notification latency is under 5 seconds.
Requirements	2.1.2
Priority	High
Estimated Time Needed	5 Minutes
Dependency	EA.FA.01 and EA.ND.01 should pass.
Setup	Edge controller is running. Mobile app is installed.
Procedure	[A01] Record the current time (T1). [A02] Trigger the MQ-2 gas sensor. [V01] Observe the mobile device for the push notification arrival and record time (T2). [V02] Calculate the latency: T2 - T1. [V03] Verify the latency is under 5 seconds.
Cleanup	Clear the gas condition.

6.27 EA.ND.02

TC_ID	EA.ND.02
Purpose	Verify that notification content includes event type and timestamp.
Requirements	2.4.1
Priority	Medium
Estimated Time Needed	3 Minutes
Dependency	EA.ND.01 should pass.
Setup	Mobile app is installed and user is logged in.
Procedure	[A01] Trigger a critical event. [A02] Wait for the push notification to arrive. [V01] Expand the notification and read its content. [V02] Verify the notification body includes the event type. [V03] Verify the notification includes a timestamp or time reference.
Cleanup	Dismiss the notification.

6.28 FR.MC.02

TC_ID	FR.MC.02
Purpose	Verify that the motion cooldown prevents excessive captures.
Requirements	2.2.1
Priority	Medium
Estimated Time Needed	5 Minutes

Dependency	FR.MC.01 should pass.
Setup	The edge controller is running.
Procedure	[A01] Trigger the PIR sensor to cause a first burst capture. [A02] Immediately trigger the PIR sensor again within the cooldown period. [V01] Verify that the first burst capture occurs normally. [V02] Verify that the second trigger does NOT start a new burst capture. [V03] Wait for the cooldown to expire, trigger again, and verify a new capture occurs.
Cleanup	None.

6.29 FR.FD.02

TC_ID	FR.FD.02
Purpose	Verify that frames without faces are correctly skipped.
Requirements	2.2.2
Priority	Medium
Estimated Time Needed	5 Minutes
Dependency	FR.MC.01 should pass.
Setup	Edge controller is running. No person is in front of the camera.
Procedure	[A01] Trigger motion detection by moving an object (not a person). [V01] Observe edge logs for the burst capture. [V02] Verify the log reports “No face in burst — skipping cloud upload.” [V03] Verify that no security event or snapshot upload is triggered.
Cleanup	None.

6.30 FR.FM.03

TC_ID	FR.FM.03
Purpose	Verify that the match score is below the threshold (0.55) for authorized persons.
Requirements	2.2.2
Priority	Medium
Estimated Time Needed	5 Minutes
Dependency	FR.FM.01 should pass.
Setup	A registered resident exists. Edge controller is running.
Procedure	[A01] The registered resident stands in front of the camera. [A02] Trigger motion detection. [V01] Observe the edge logs for the face matching result. [V02] Extract the Euclidean distance score from the log. [V03] Verify that the score is less than or equal to 0.55.
Cleanup	None.

6.31 FR.SU.02

TC_ID	FR.SU.02
Purpose	Verify that unauthorized detection triggers an intrusion alert notification.
Requirements	2.2.2
Priority	High

Estimated Time Needed	5 Minutes
Dependency	FR.FM.02 and EA.ND.01 should pass.
Setup	Edge controller is running. Mobile app is installed.
Procedure	[A01] An unregistered person stands in front of the camera. [A02] Trigger motion detection. [V01] Verify a “stranger_detected” security event is created. [V02] Verify a push notification is received on the mobile device. [V03] Verify the notification content indicates an unauthorized person was detected.
Cleanup	None.

6.32 DV.RS.02

TC_ID	DV.RS.02
Purpose	Verify that sensor values update on the dashboard when new readings arrive.
Requirements	2.3.2
Priority	Medium
Estimated Time Needed	5 Minutes
Dependency	DV.RS.01 should pass.
Setup	User logged in and dashboard open.

Procedure	[A01] Note the current temperature and humidity values displayed. [A02] Wait for the next sensor reading cycle to complete (default 60s). [V01] Verify that the displayed values are refreshed with the new reading. [V02] Verify the timestamp of the displayed reading has been updated.
Cleanup	None.

6.33 DV.HC.02

TC_ID	DV.HC.02
Purpose	Verify that selecting a date range filters the historical chart data accordingly.
Requirements	2.3.2
Priority	Medium
Estimated Time Needed	3 Minutes
Dependency	DV.HC.01 should pass.
Setup	The user is logged in and on the History screen.
Procedure	[A01] Select a specific date range using the date filter controls. [V01] Verify the chart re-renders to show only data within the selected range. [A02] Change the date range to a different period. [V02] Verify the chart updates to reflect the new range.
Cleanup	None.

6.34 DV.CE.02

TC_ID	DV.CE.02
Purpose	Verify that the face classification label is shown for camera events.
Requirements	2.2
Priority	Medium
Estimated Time Needed	3 Minutes
Dependency	DV.CE.01 should pass.
Setup	Camera events with both classifications exist.
Procedure	[A01] Open the Camera screen. [V01] Locate a camera event where the person was classified as a resident. [V02] Verify the label displays the resident’s name. [V03] Locate a camera event where the person was classified as unknown. [V04] Verify the label displays “Unknown” or “Unauthorized”.
Cleanup	None.

6.35 RM.AR.02

TC_ID	RM.AR.02
Purpose	Verify that a resident can be added without a photo.
Requirements	2.4.3
Priority	Medium
Estimated Time Needed	3 Minutes
Dependency	User must be authenticated.

Setup	The user is on the Residents screen.
Procedure	[A01] Navigate to the Residents screen. [A02] Tap/click “Add Resident”. [A03] Enter only the resident’s name without uploading a photo. [A04] Confirm the addition. [V01] Verify the new resident appears in the residents list. [V02] Query the Supabase residents table and confirm the entry exists with a null photo_path. [V03] Verify the embedding field remains null (no photo to compute from).
Cleanup	Delete the test resident if needed.

Conclusion

Component	Technology / Implementation
Edge Device	Raspberry Pi 5 (8 GB RAM)
Backend	FastAPI + Supabase
Frontend	React 19 + Vite
Mobile Application	Flutter
AI Pipeline	MediaPipe + dlib Face Recognition
Streaming	WebSocket Relay Architecture
Database	Supabase PostgreSQL

The Smart Home System project successfully achieved its primary objective of developing an integrated IoT platform combining environmental monitoring, AI-powered security, realtime communication, and multi-platform accessibility within a unified architecture.

During the development lifecycle, the project evolved from a prototype into a production-oriented distributed system consisting of a Raspberry Pi 5 edge controller, a FastAPI cloud gateway, a Supabase-based backend infrastructure, a React web dashboard, and a Flutter mobile application.

One of the strongest aspects of the system was the implementation of an edge-first architecture. Instead of transferring all workloads to the cloud, the Raspberry Pi performed local sensor processing and face recognition directly on the device. This approach reduced latency, minimized bandwidth consumption, and improved overall system resilience.

The AI-based face recognition pipeline became one of the major technical contributions of the project. By combining MediaPipe face detection with a fallback dlib-based recognition mechanism, the system achieved stable recognition performance on ARM-based hardware under standard indoor conditions.

> “More importantly, the project evolved beyond a simple academic prototype and became a production-oriented distributed IoT platform capable of solving real-world smart home monitoring and security problems.”

Key Achievements

- Realtime sensor monitoring using Supabase Realtime subscriptions
- Hybrid AI face recognition pipeline with MediaPipe and dlib
- On-demand WebSocket live streaming architecture
- Offline-resilient synchronization system using SQLite queueing
- Role-based authentication and cloud synchronization
- Cross-platform support through React web and Flutter mobile applications
- Stable long-duration runtime with realtime synchronization

System Stability and Testing

System testing demonstrated that the platform can operate continuously for extended durations under real-world conditions. During development and evaluation, the system remained active for sessions lasting up to approximately 12 hours continuously and was actively tested throughout nearly two months of iterative development. Although a small number of crashes were observed during early iterations, these issues were significantly reduced through debugging, synchronization improvements, and architectural refinements. By the final implementation stage, the realtime infrastructure and core monitoring systems were operating in a stable and reliable manner.

Future Work

Despite the successful implementation of the current system, several opportunities remain for future improvement and expansion.

Future Area	Description
Multi-Camera Support	Support for multiple synchronized camera streams and centralized monitoring.
Smart Automation	Implementation of advanced automation scenarios such as Away Mode and Night
Energy Monitoring	Integration of power consumption sensors for realtime energy analytics.
Voice Assistant Integration	Compatibility with Google Assistant, Alexa, or local voice assistants.
Additional Sensors	Support for CO, air quality, vibration, and water leakage sensors
Security Improvements	Migration from ws:// to wss:// with TLS encryption and stronger device authenticat
AI Improvements	Use of Edge TPU acceleration and ANN indexing methods such as FAISS.

In conclusion, this project demonstrates the feasibility of building a modern, intelligent, and distributed smart home ecosystem using affordable hardware and contemporary cloud technologies. The platform combines IoT sensor integration, edge-based artificial intelligence, realtime communication, and cross-platform accessibility into a cohesive and functional system. The experience gained throughout the development process provided valuable insight into distributed systems, embedded software engineering, computer vision, cloud infrastructure, and realtime application development, establishing a strong foundation for future academic and professional work.

References

[1] Albany, M. (2022). *A review: Secure Internet of Thing System for Smart Houses*. [Conference/journal details].

[2] Bouchabou, D., Nguyen, S. M., Lohr, C., Leduc, B., & Kanellos, I. (2021). A Survey of Human Activity Recognition in Smart Homes Based on IoT Sensors Algorithms: Taxonomies, Challenges, and Opportunities with Deep Learning. *arXiv preprint* arXiv:2111.04418.

[3] Sutar, P., Sarkar, S., Tagare, A., & Pawar, A. (2024). A Review on IoT-Enabled Smart Homes Using AI. *IEEE ICCCNT*.

[4] Zia, M. F., Siddiqua, M., Ouameur, M. A., Bagaa, M., & Al Turjman, F. (2025). Securing the Future: A Survey on Smart Home Security in IoT-Integrated Smart Cities. *Advances in Networks, 12*(1), 1–18.

[5] R. Harper, "The Connected Home: A Comparative Review of IoT Ecosystems," International Journal of Smart Home Technology, vol. 12, no. 3, pp. 45-58, 2023.

[6] J. Smith and A. Doe, "Architectural Trade-offs in Modern Smart Home Platforms: Cloud vs. Edge Computing," IEEE Internet of Things Journal, vol. 10, no. 14, 2024.

[7] Samsung Developers, "SmartThings Architecture and Hub Connectivity," Samsung Developer Documentation, 2023. [Online]. Available: developer.smartthings.com.

[8] Samsung SmartThings, "Edge Drivers: Moving Intelligence to the Edge," Technical White Paper, Oct. 2022.

[9] Connectivity Standards Alliance (CSA), "The Matter Standard: Universal IP-based Connectivity for Smart Home," CSA Specification Version 1.2, 2023.

[10] Samsung Knox, "Security Solutions for IoT and Smart Home," Samsung Enterprise Security, 2023.

[11] Apple Inc., "Platform Security Guide: HomeKit Security and Privacy," Apple Support Documentation, May 2024.

[12] Apple Inc., "HomeKit Secure Video: Local Analysis and Encryption Standards," Apple Developer Archive, 2023.

[13] Google Nest, "Google Home Platform Architecture: Cloud-to-Cloud Integrations," Google Developers, 2023. [Online]. Available: developers.home.google.com.

[14] Google Developers, "Building for Matter on Google Home," Google I/O Technical Sessions, 2023.

[15] M. Johnson, "Data Collection and Privacy in Voice Assistant Ecosystems," Journal of Cybersecurity and Privacy, vol. 5, no. 1, pp. 112-129, 2023.

[16] Home Assistant, "The State of the Open Home: Local Control and Privacy," Home Assistant Official Documentation, 2024. [Online]. Available: home-assistant.io.

[17] P. Schiestl, "Zigbee and Z-Wave Integration in Open Source Home Automation," Proceedings of the Open Source IoT Conference, 2022.

[18] Electronic Frontier Foundation (EFF), "Privacy Analysis of Consumer IoT Devices," Tech Policy Report, 2023.